Mini Project 2
Mini Chess AI

Due: 6/20

Demo: 6/21 online

Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

Introduction

- Design and implement an AI which can play MiniChess
- Read the current board and output the next move
- Design a state value function to evaluate the score of the board
- Determine the next move with tree search algorithm

Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

Chess

Mini Chess

- Has lot of variance.
- We use “**MinitChess**” in this project

Basic rules

- We have 2 players: white and black. White plays first.
- If the target place of your piece has an opponent's piece, you can take it out (or, catch the piece). You cannot take your own piece.
- If a player can catch the opponent's king in its turn, it wins.
  - So a player can only win in their turn or lose in the opponent's turn.
- If a player makes an illegal move, he/she loses.

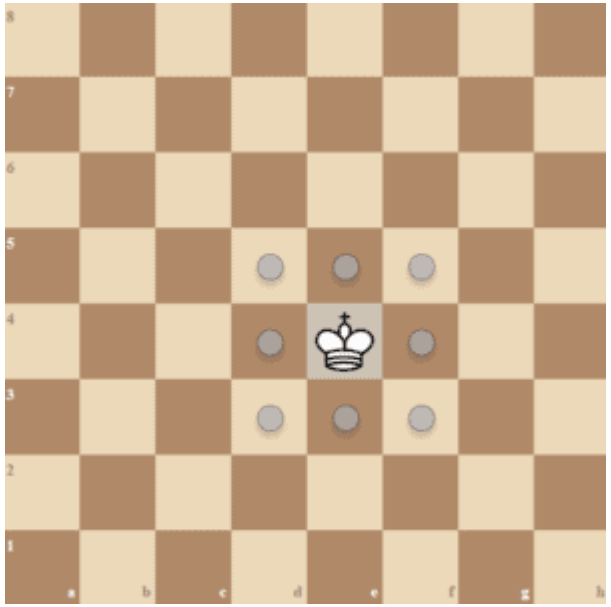
- Our rule is simplified; any different rules will be marked in red:
- There is no castling
- Pawn can only promote to Queen (detail in the next section)

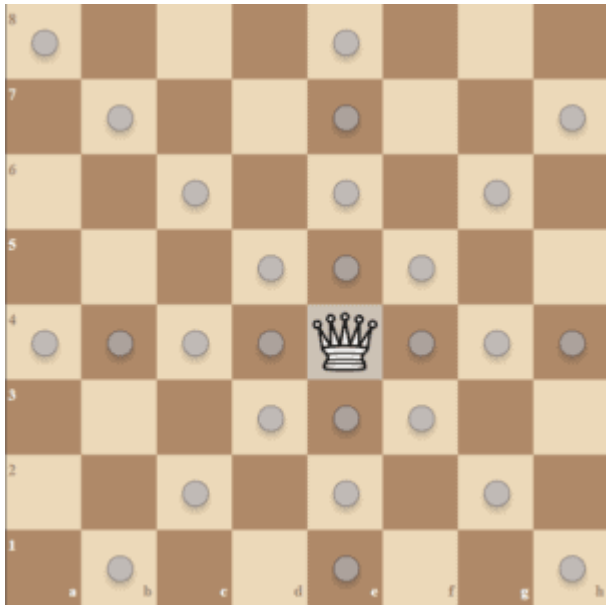
Basic rules

- If 2 players have almost the same ability, it is very likely to get a draw and fall into an infinite loop. So we add these rules:
  - If it cost over 50 step(25 turn), we count the piece value.
    Queen=20, Bishop=8, Knight=7, Rook=6, Pawn=2.
    The player with the higher piece value after 50 steps wins.
  - If both side has same value, it is a draw.
  - The Queen, which is promoted by Pawn, counts as a queen.

Piece Movement

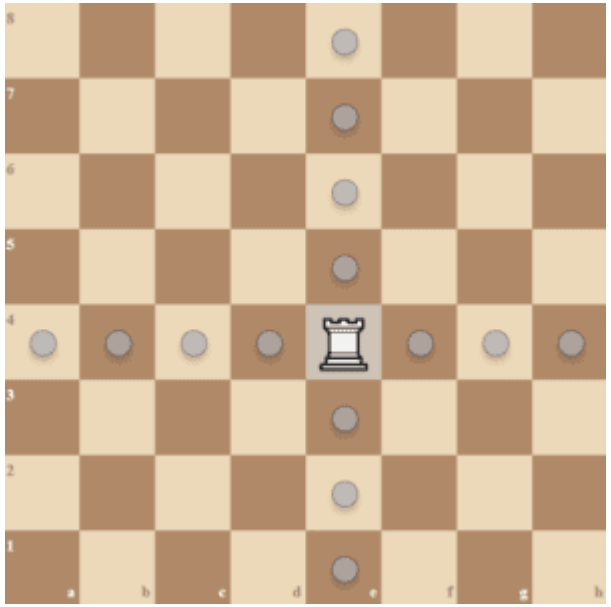
- King and Queen

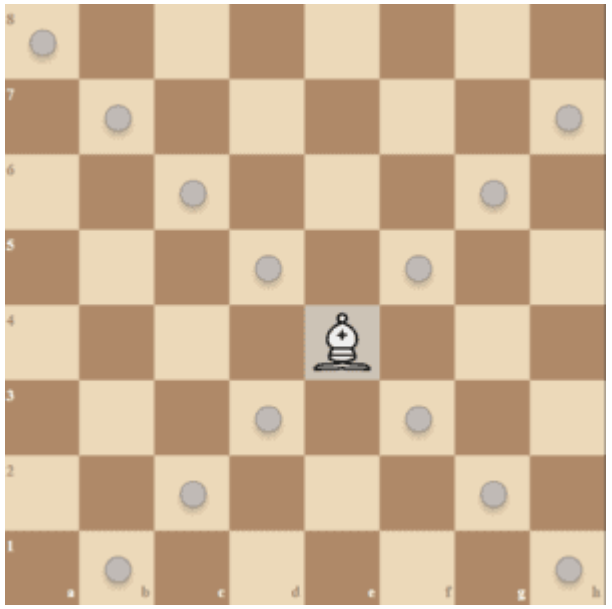




Piece Movement

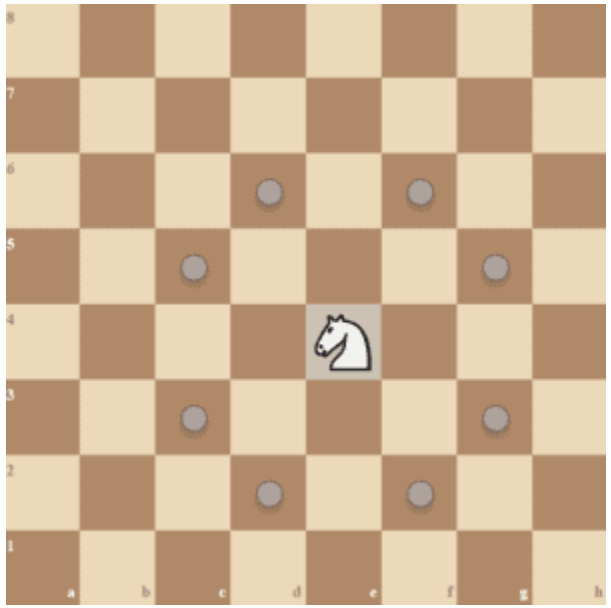
- Rook and Bishop



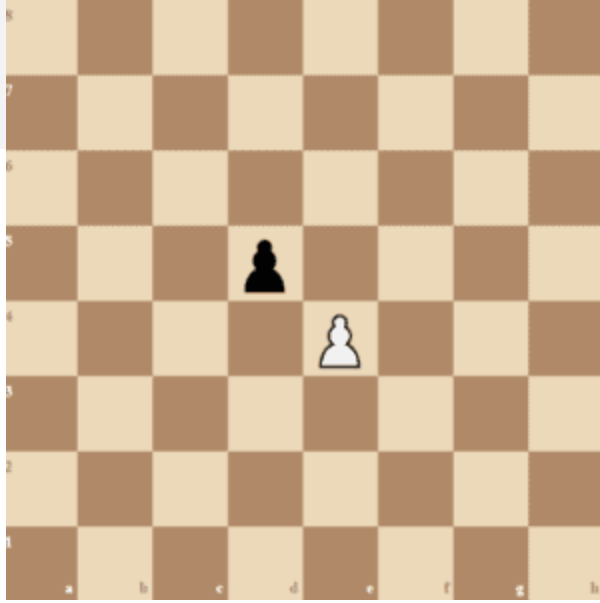


Piece Movement

- Knight
- Knight can “jump over pieces”
  - It cannot be blocked



Piece Movement



- Pawn
- Every turn, pawn can move forward one step.
  - Even in the first move of that pawn.
  - With ↑ this rule, there is no En passant.
- If left/right forward is the opponent's piece, you can catch it.
- When a pawn moves to the last row (6 for white, 1 for black), it becomes Queen (promotion).
  - Promotion and Move to Last Row will happen simultaneously.

Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

State Value Function

- The program should decide which move is better
- We can pick the move that leads to the board with the highest score
- Thus, we need a function to evaluate the score of the board
- It is the “state value function”

State Value Function

- State => the board
- Value => how “good” the board is
- Function => given a board, output the value

State Value Function

- Super simple Example:
- Give every piece a score (king=inf, queen=100, …)
- Your pieces – Opponent’s pieces = value of the state.

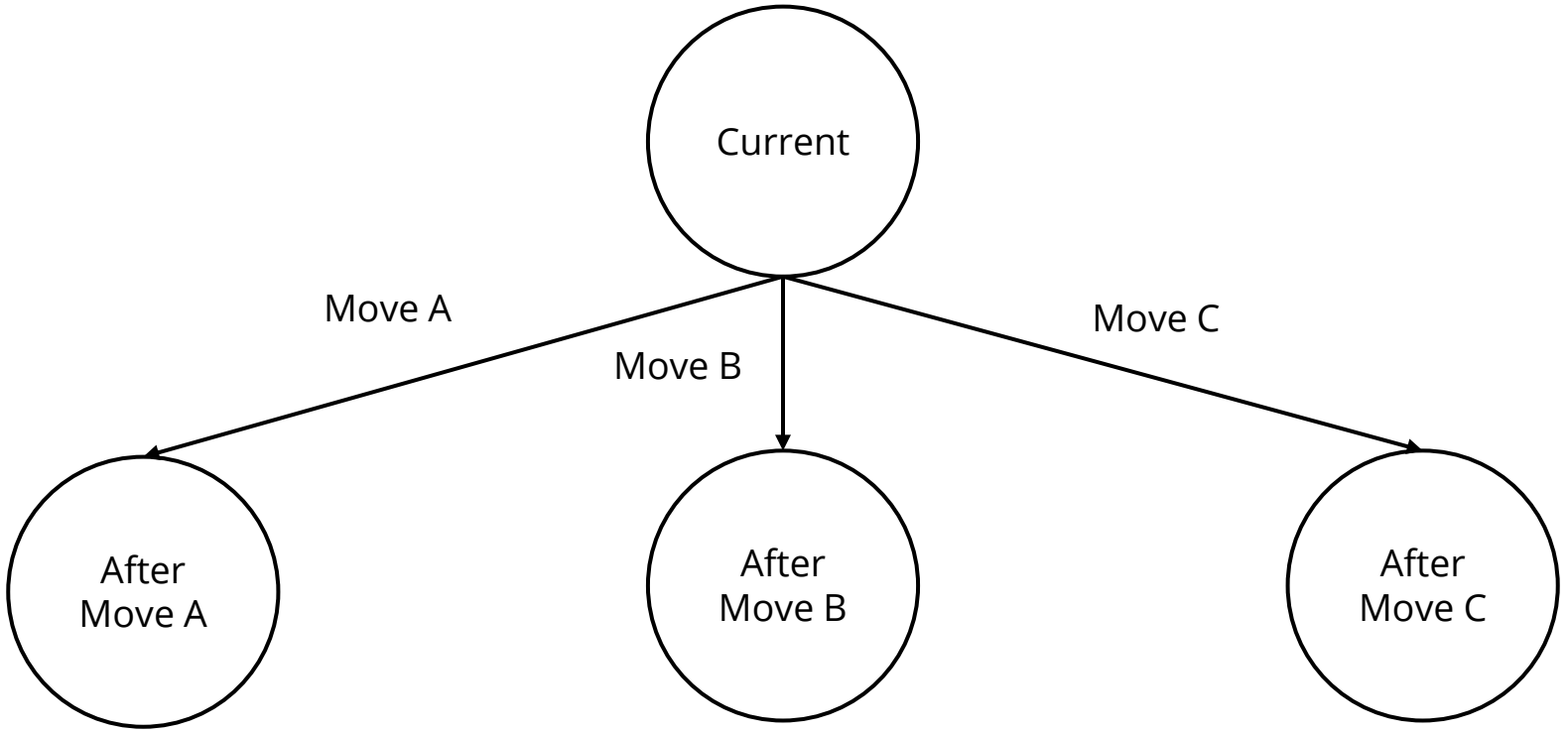
- Some upgrade:
- A piece in a different place has a different value.

State Value Function

- Keywords for more complicated algorithm:
- KP (King-Piece), PP (Piece-Piece), KPPT (King-Piece-Piece-Turn)
- KKPT (King-King-Piece-Turn with King-Piece-Piece)
- MCTS (Monte Carlos Tree Search)
- AlphaZero
  - Leela Chess Zero
  - DLShogi
- NNUE (Efficiently Updatable Neural Network)
  - This is the SOTA for Chess and Shogi
  - Stockfish (2022 TCEC 1st place, 2022 CCC 1st place)
  - 水匠 (第3回世界将棋AI電竜戦優勝)

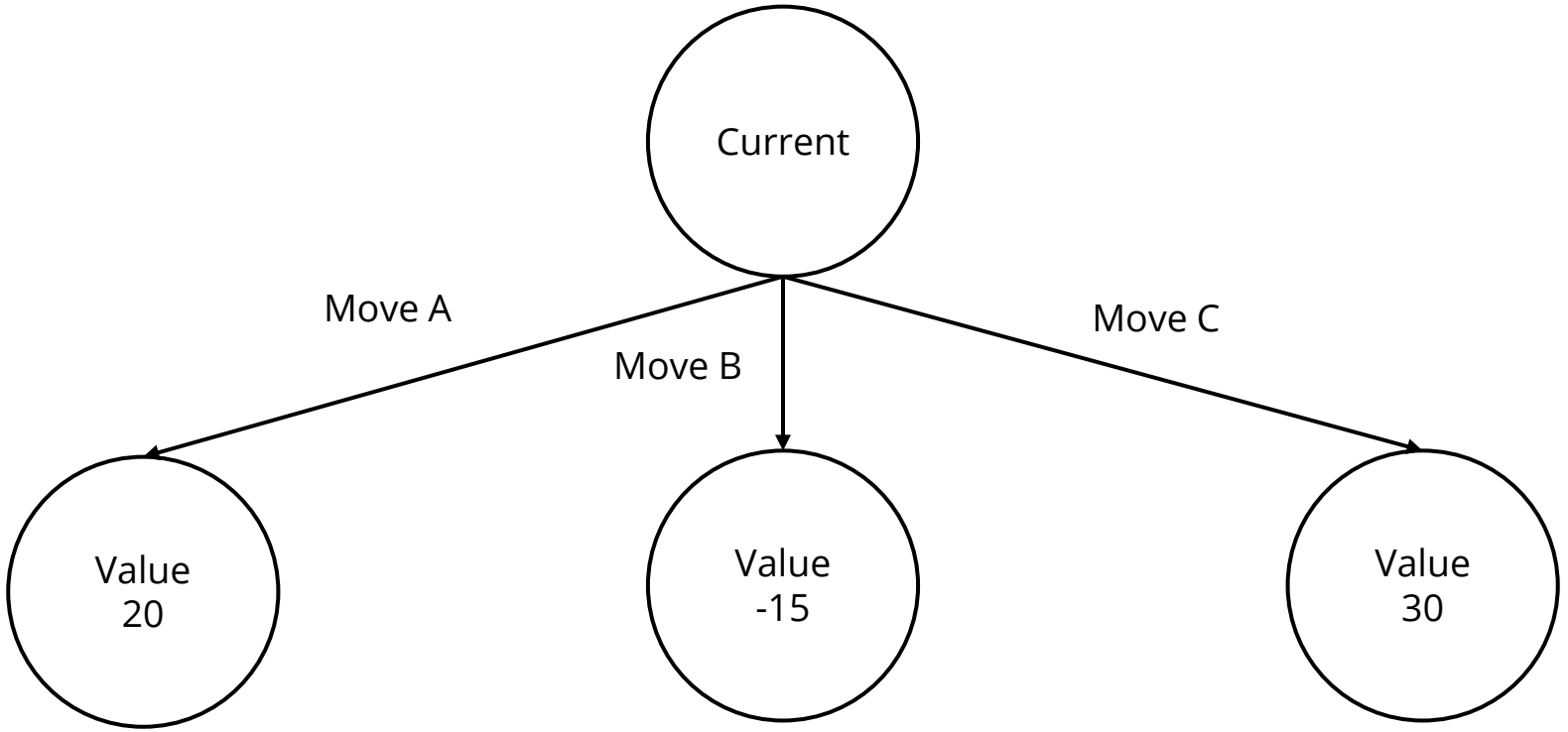
Use value function to pick the next move

- Suppose we have three valid moves, A, B, and C:



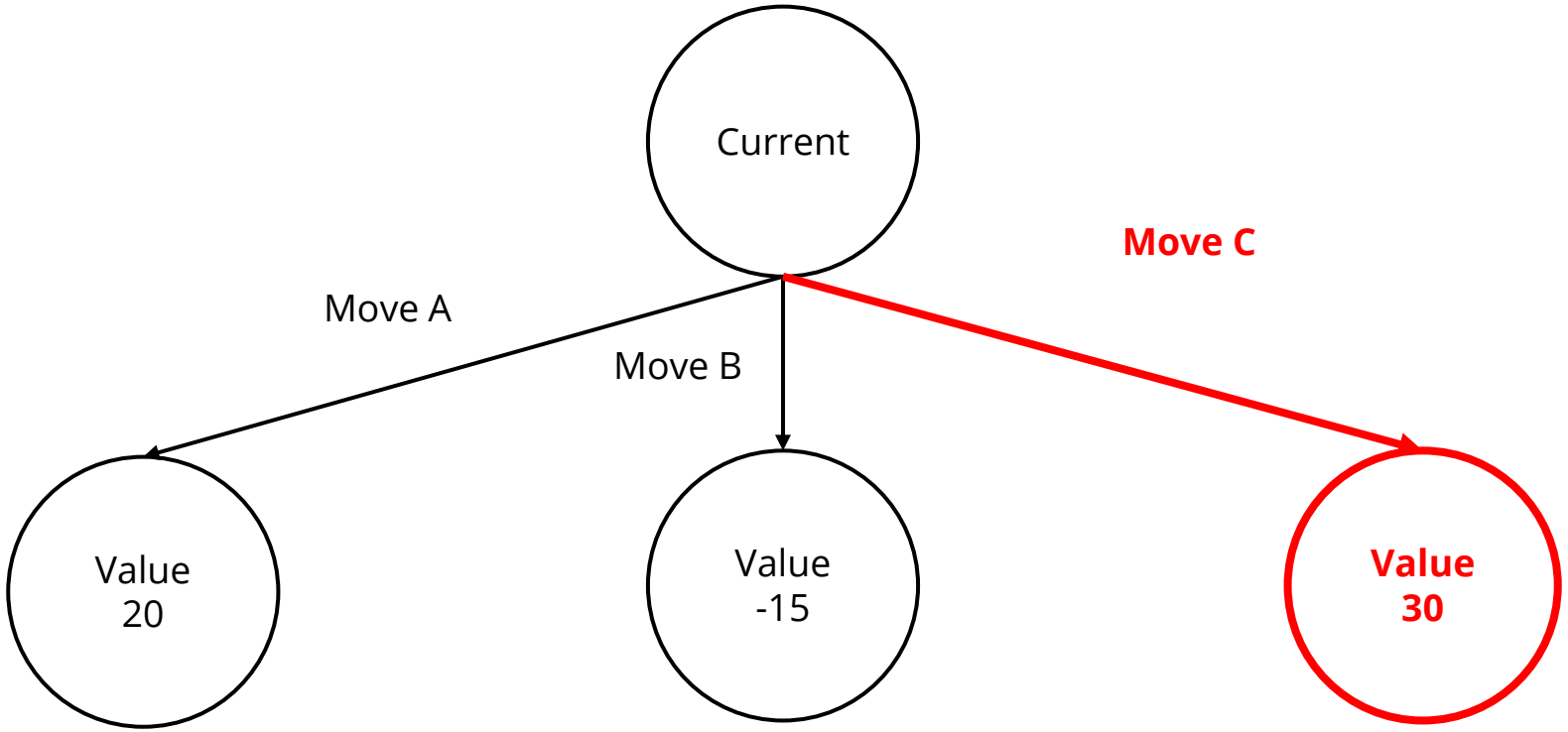
Use value function to pick the next move

- After evaluating the state values, we have 20, -15, and 30



Use value function to pick the next move

- We pick move C to be our next step since it leads to the highest value



Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

Minimax

- In the previous example, we only look forward for one step
- However, the opponent will try its best to defeat you
- Greedy choice is not always the best
- We should look forward to more steps and simulate how the opponent thinks to make the best choice with the least risk

Minimax

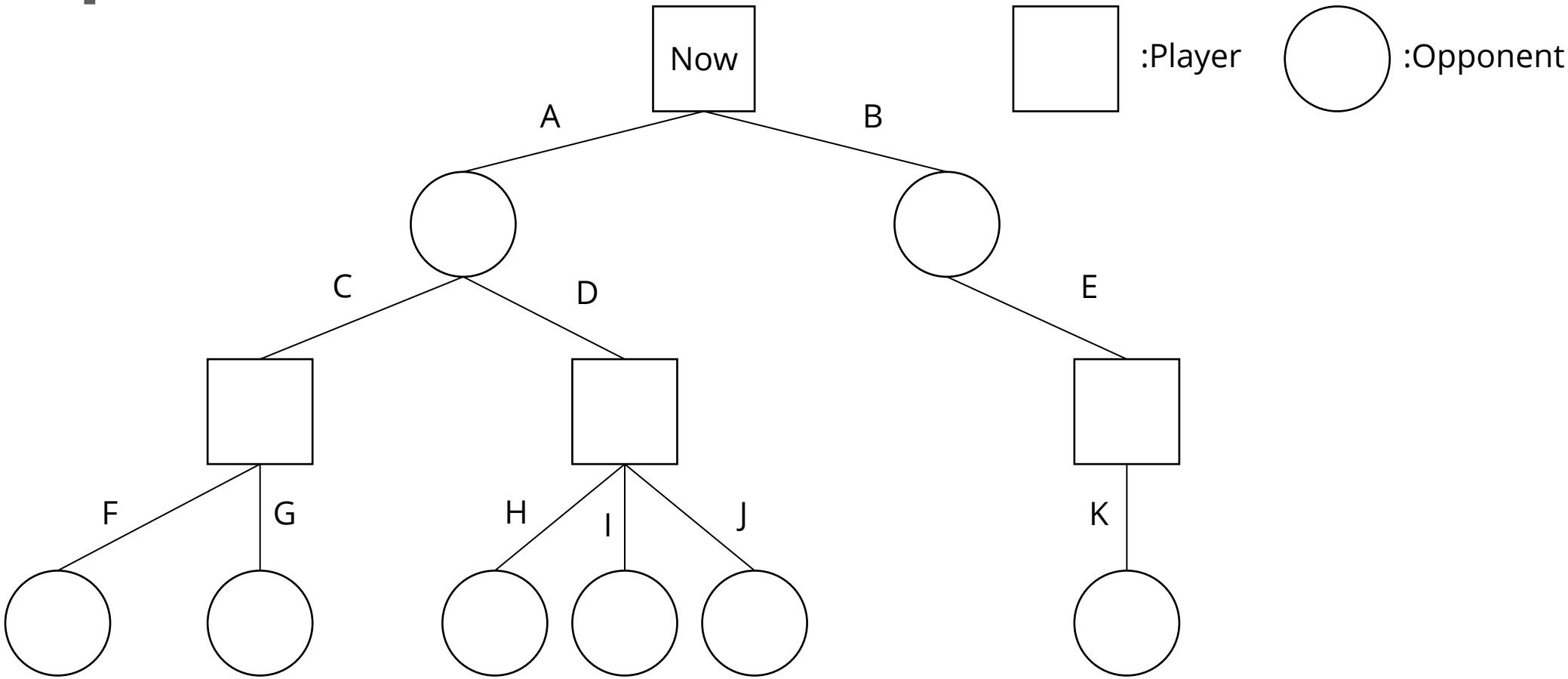
- Player tries its best to win
  - Player picks the move with the highest score
- Opponent tries its best to defeat the player
  - Opponent picks the move with the lowest “player’s value function” score
  - That is, the opponent tends to give the player the worst board
- The Minimax algorithm is based on this player-opponent interaction

Minimax Pseudocode

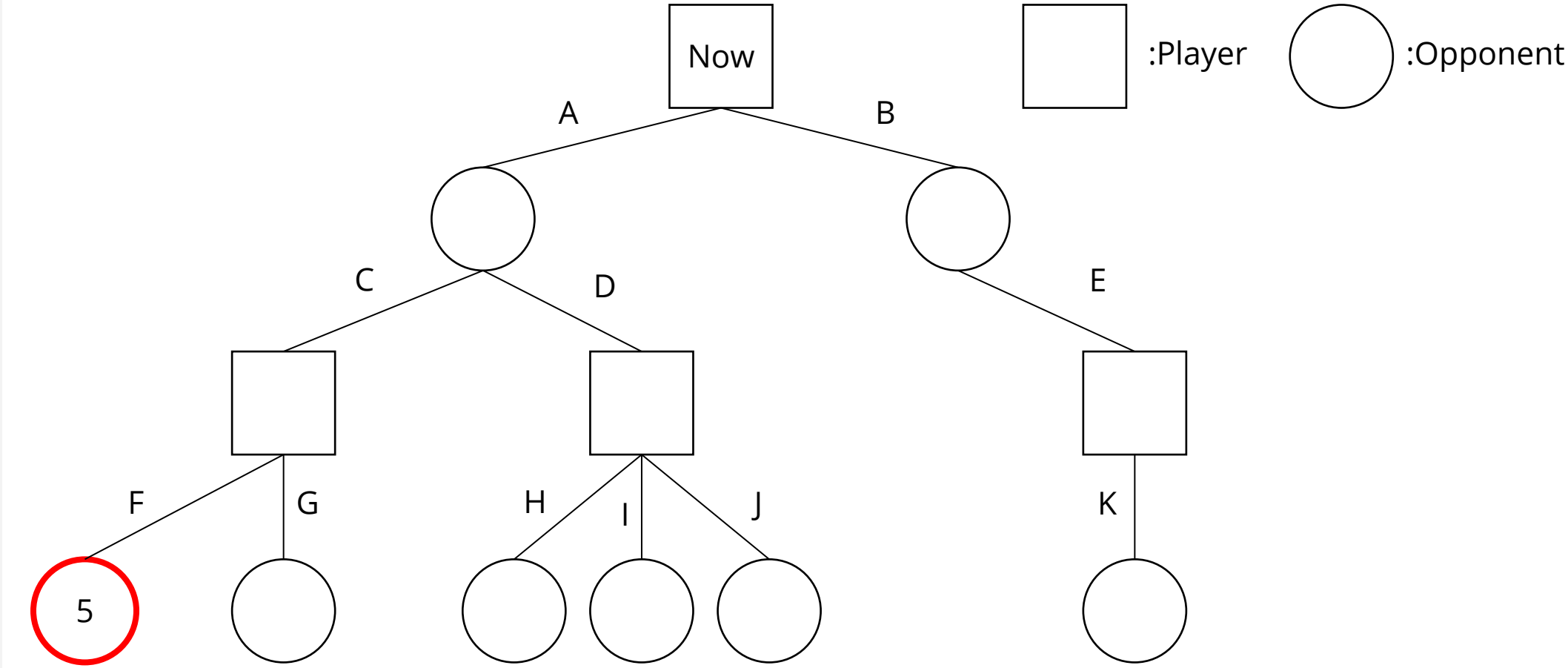
```
function minimax(node, depth, maximizingPlayer) is
    if depth = 0 or node is a terminal node then
        return the heuristic value of node
    if maximizingPlayer then
        value := −∞
        for each child of node do
            value := max(value, minimax(child, depth − 1, FALSE))
        return value
    else (* minimizing player *)
        value := +∞
        for each child of node do
            value := min(value, minimax(child, depth − 1, TRUE))
        return value
```

Source:
https://en.wikipedia.org/wiki/Minimax

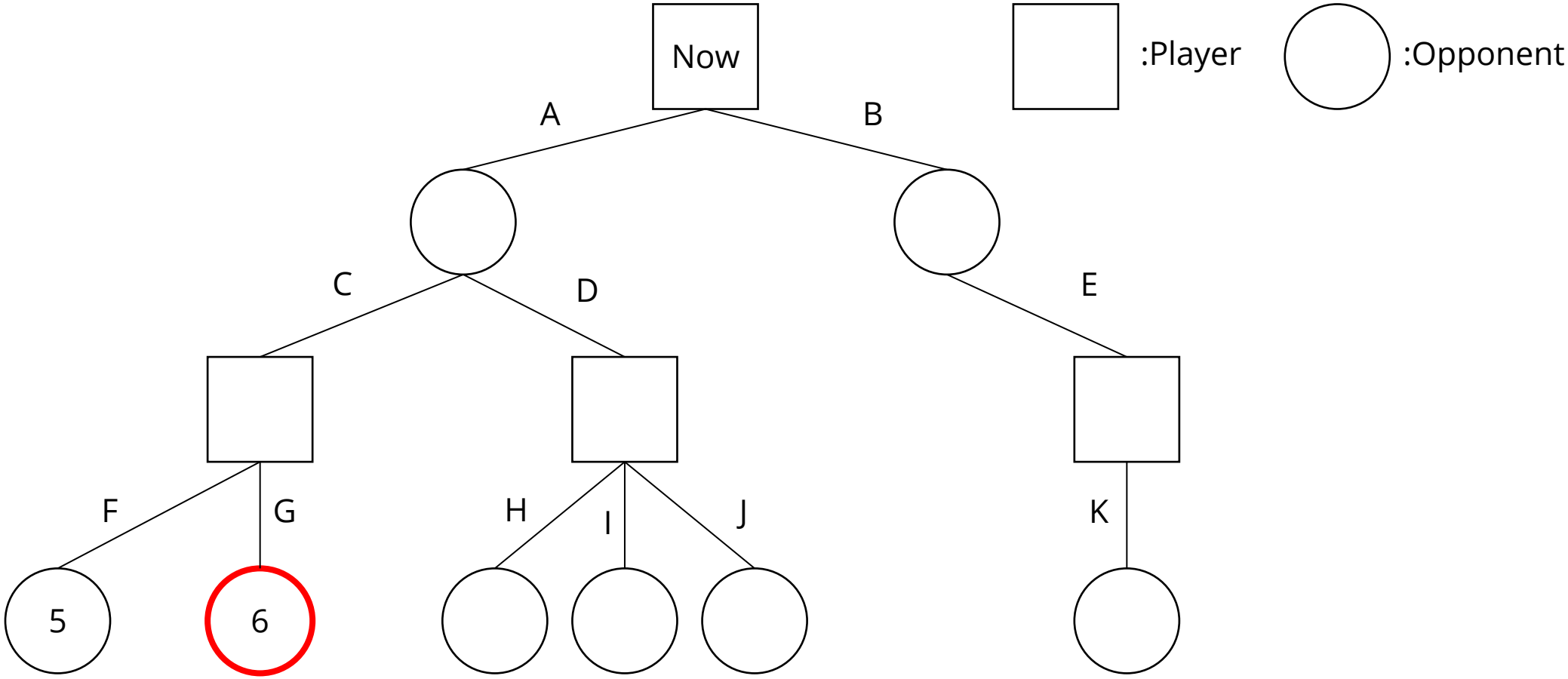
Example



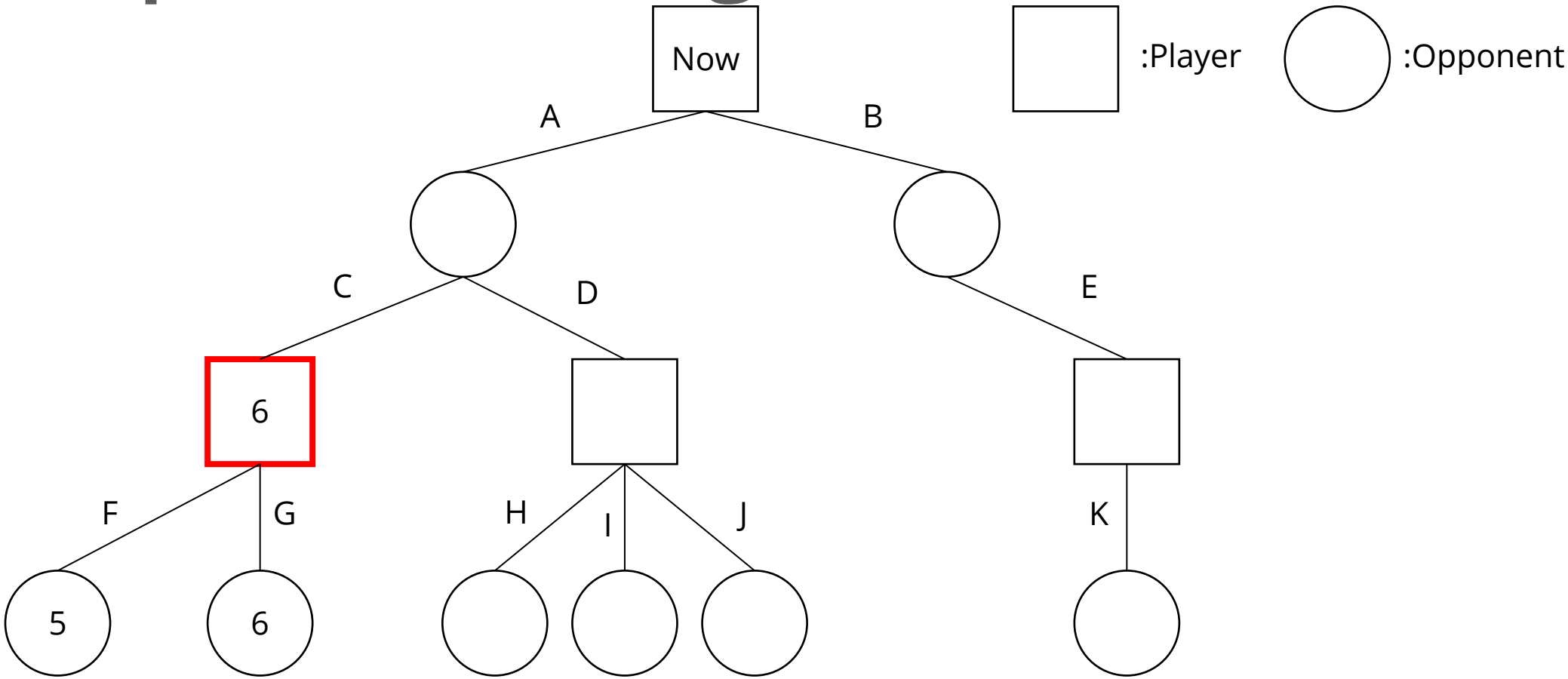
Evaluate score at leaves



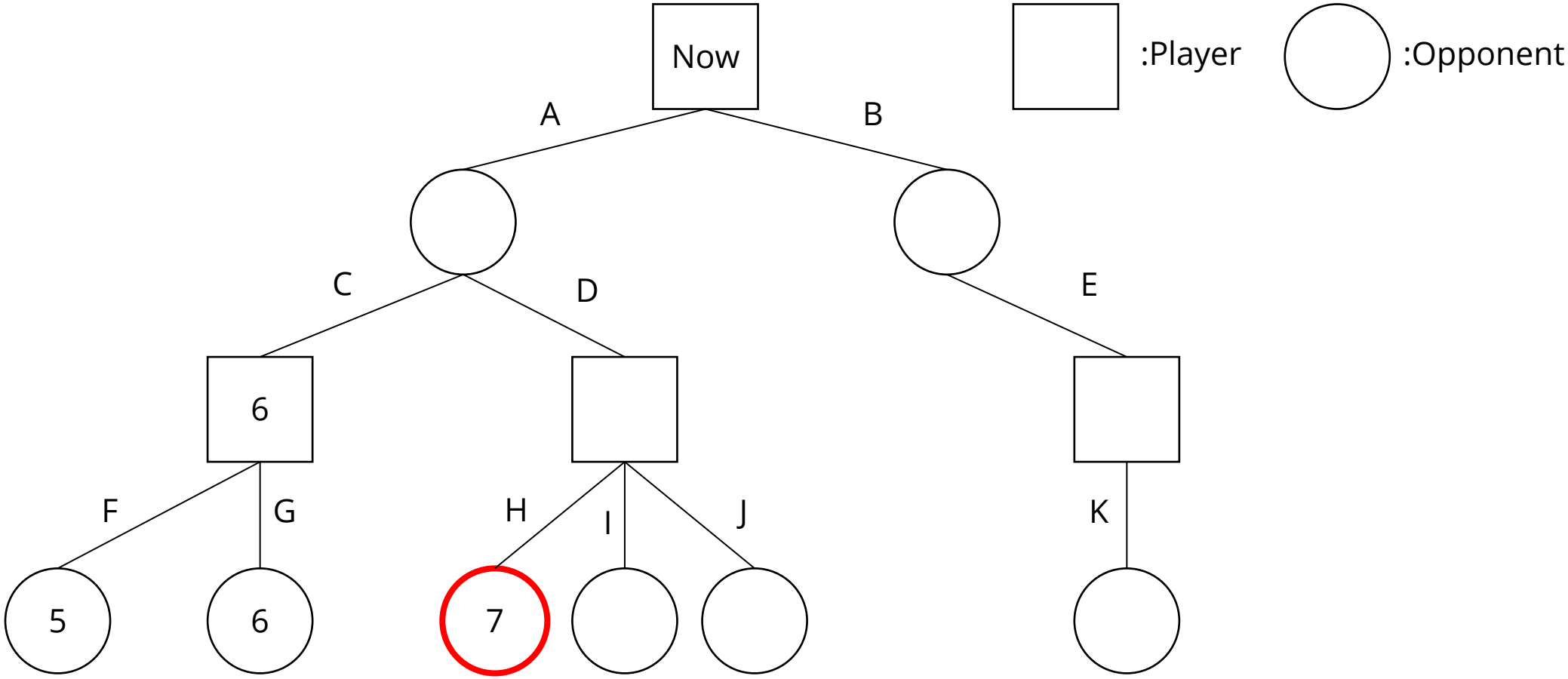
Evaluate score at leaves



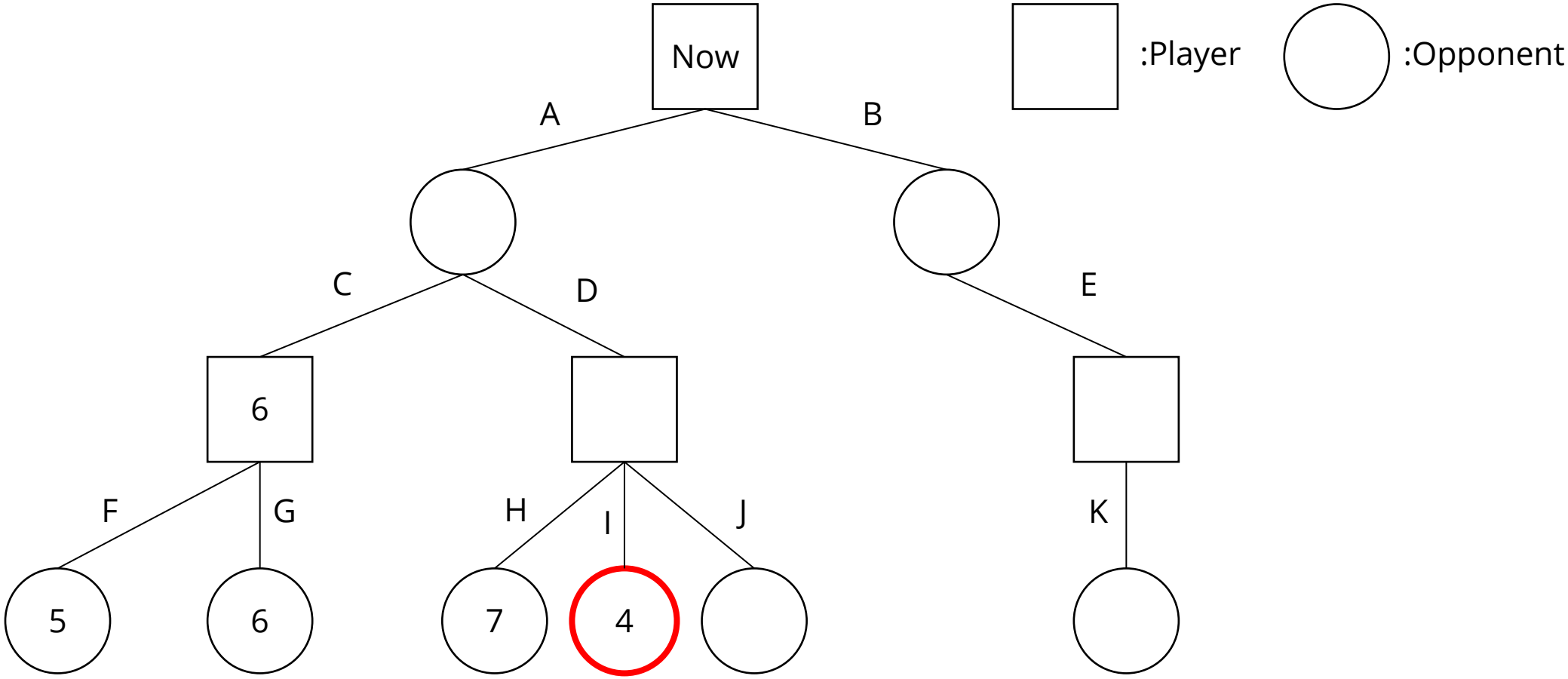
Player picks the largest score



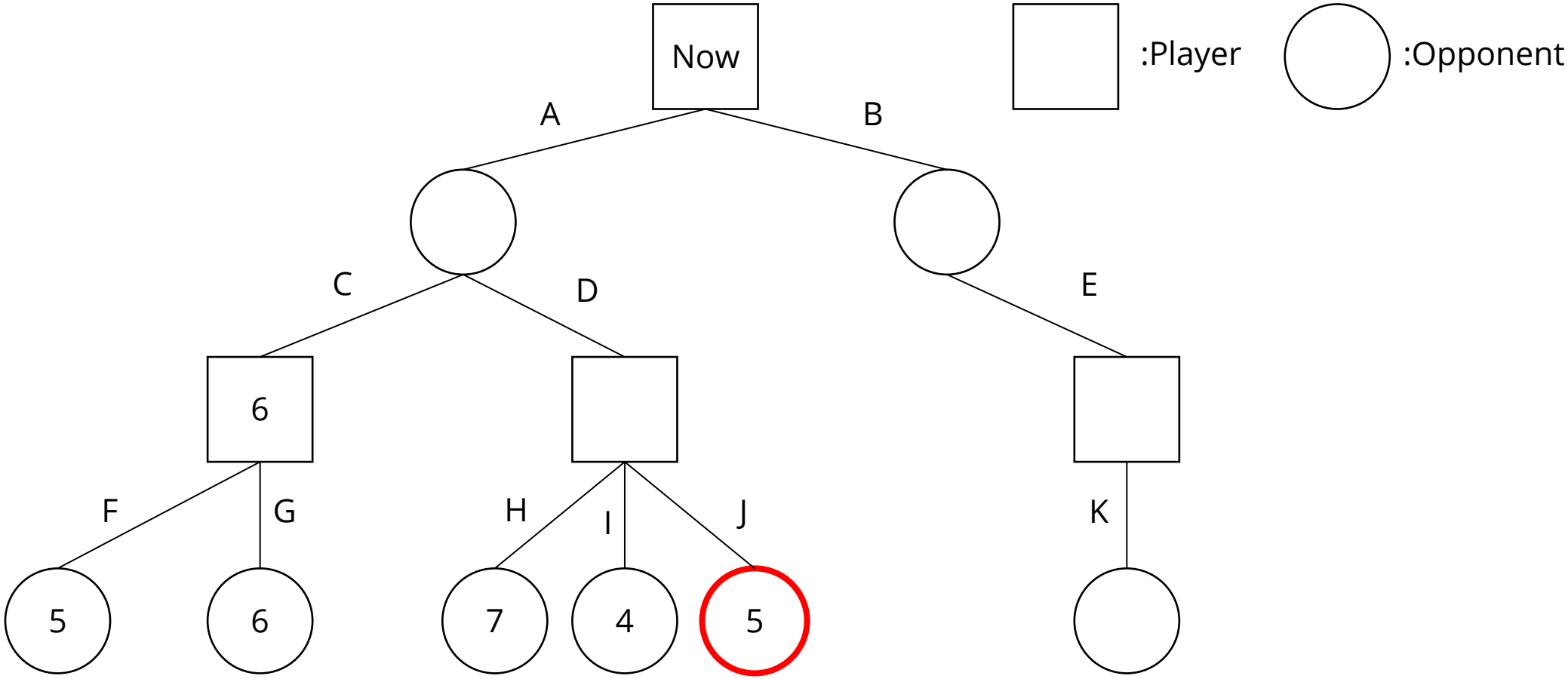
Evaluate score at leaves



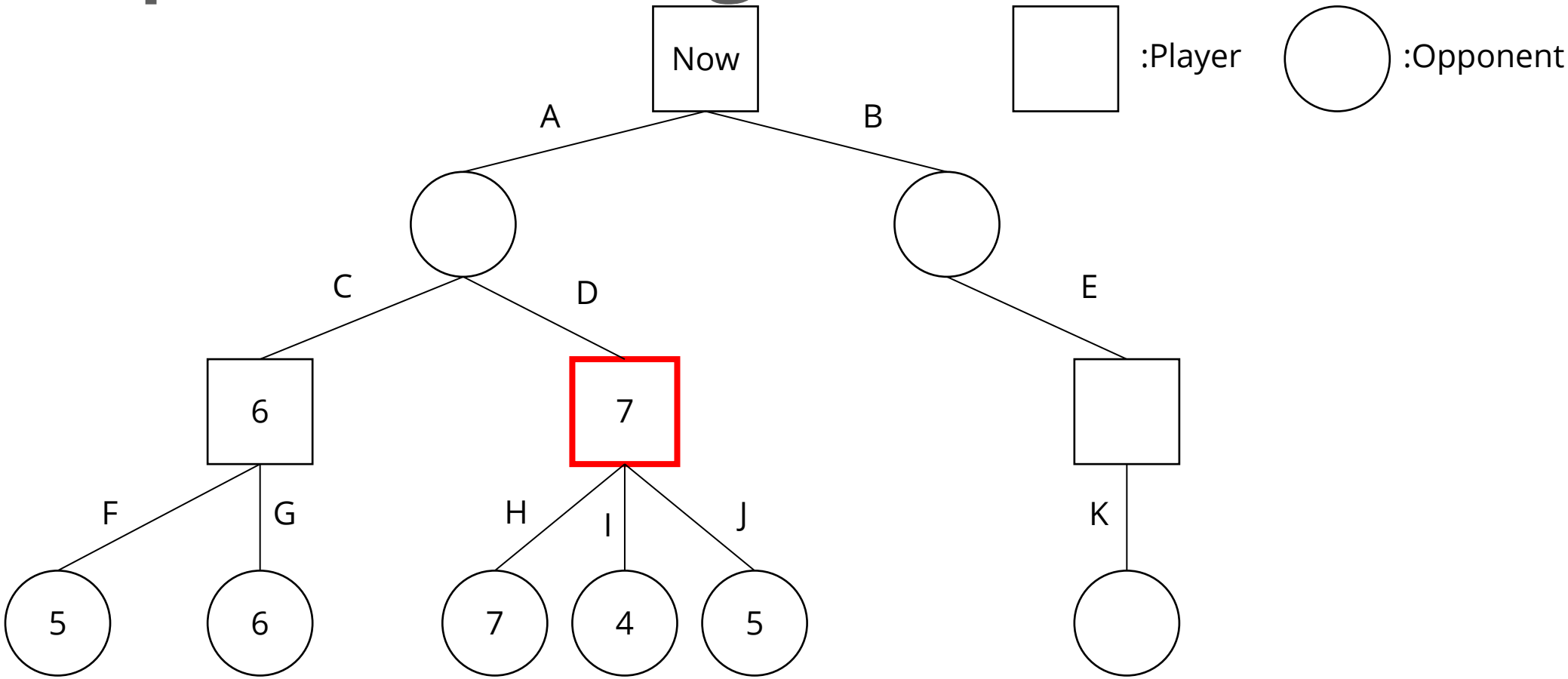
Evaluate score at leaves



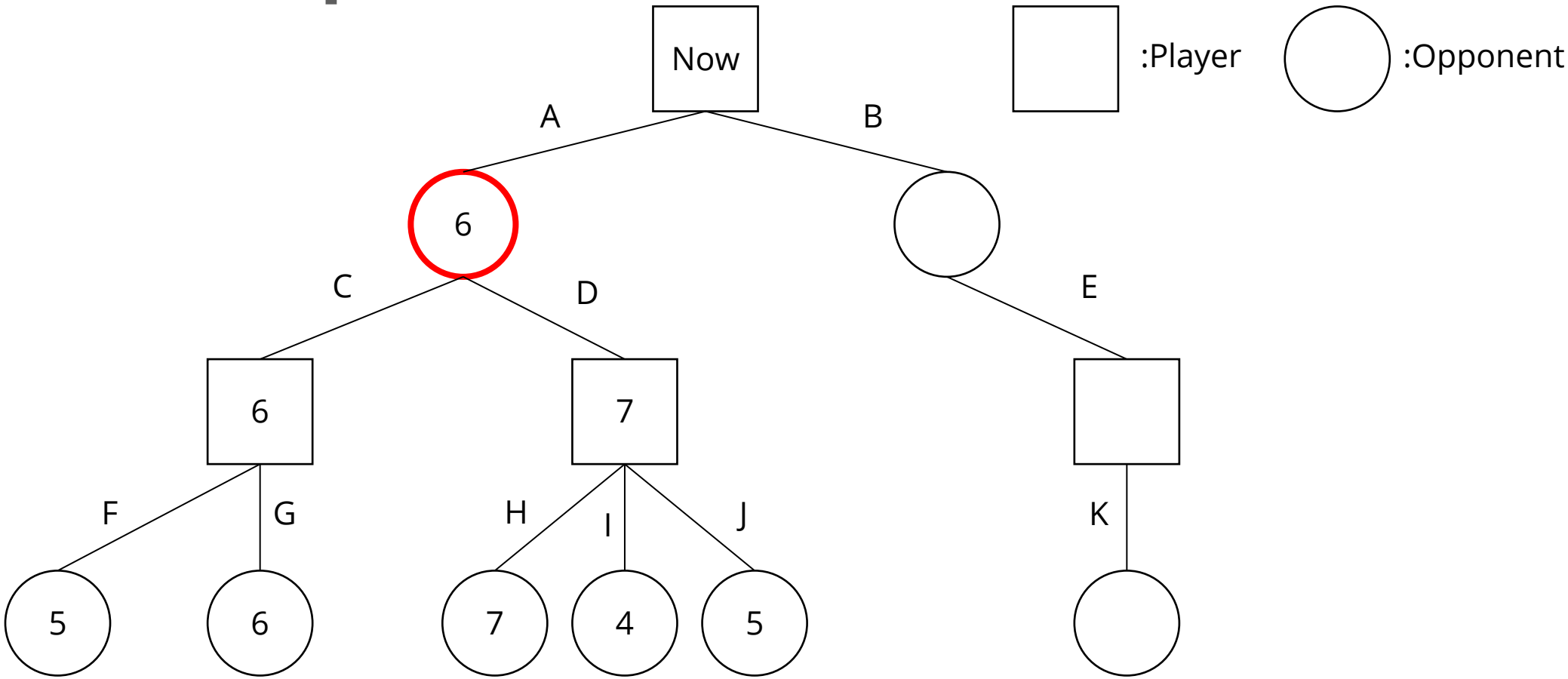
Evaluate score at leaves



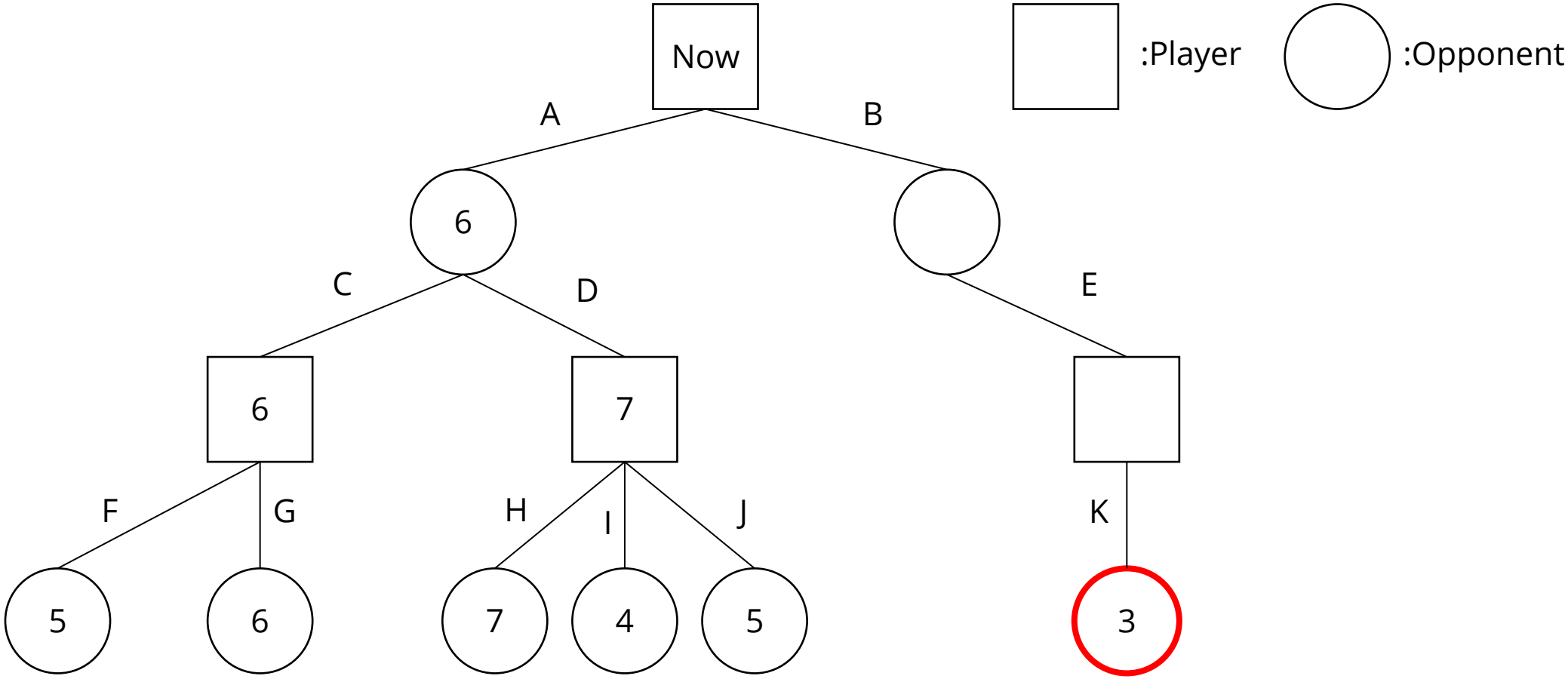
Player picks the largest score



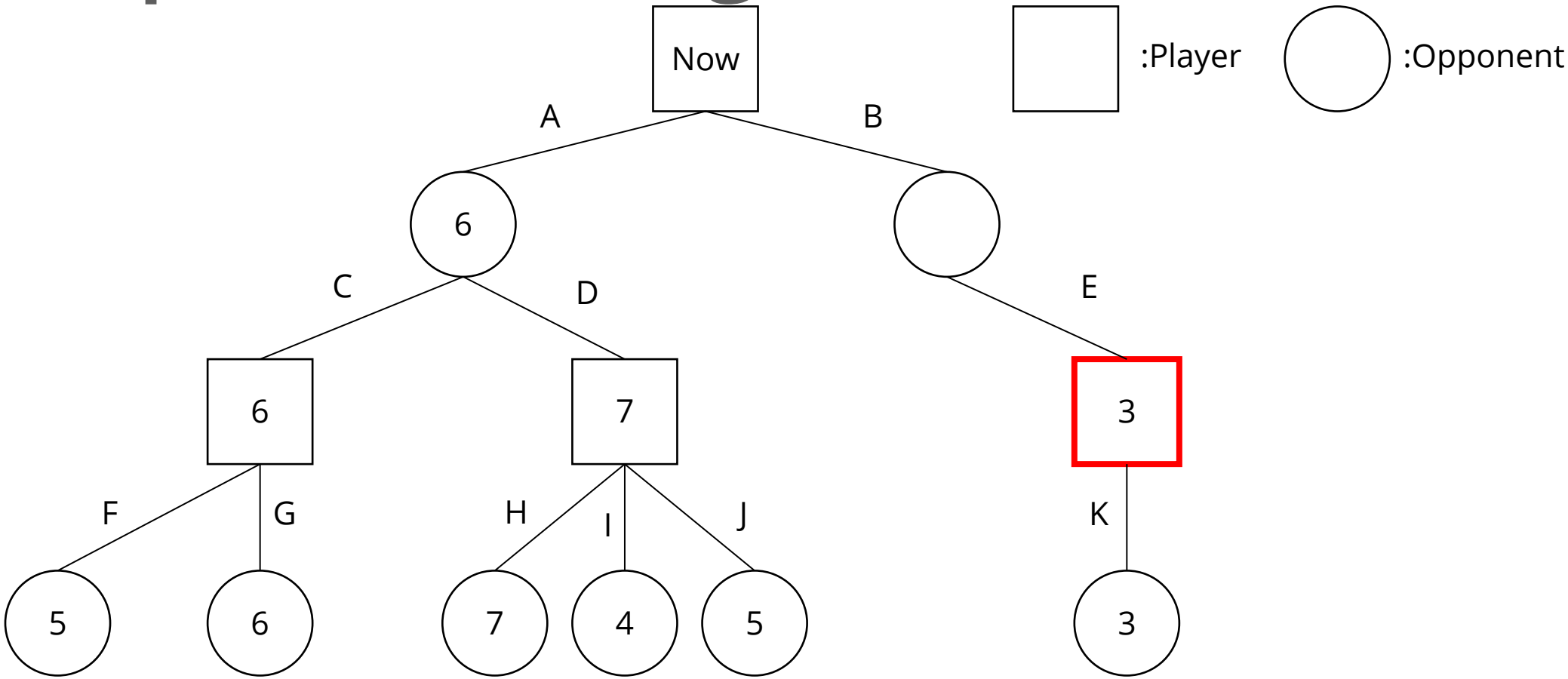
Opponent picks the smallest score



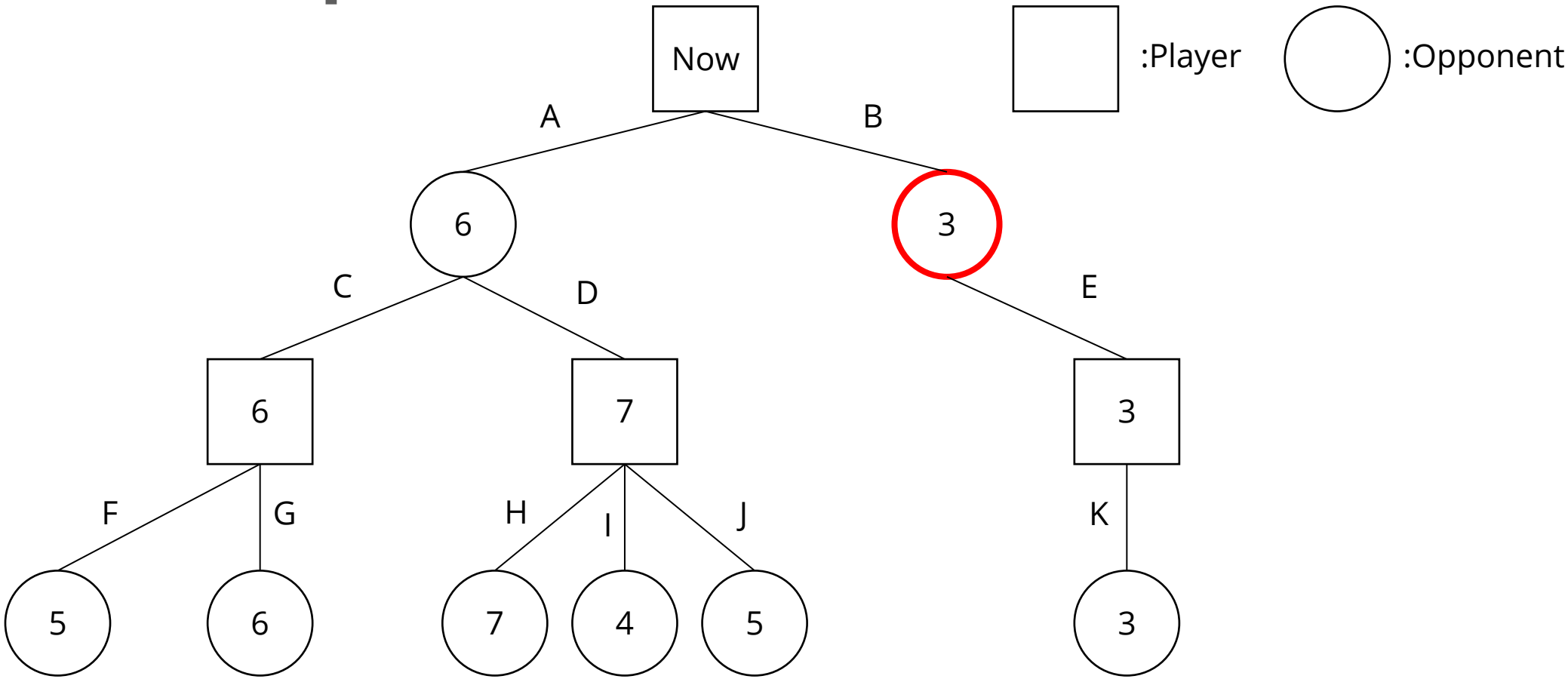
Evaluate score at leaves



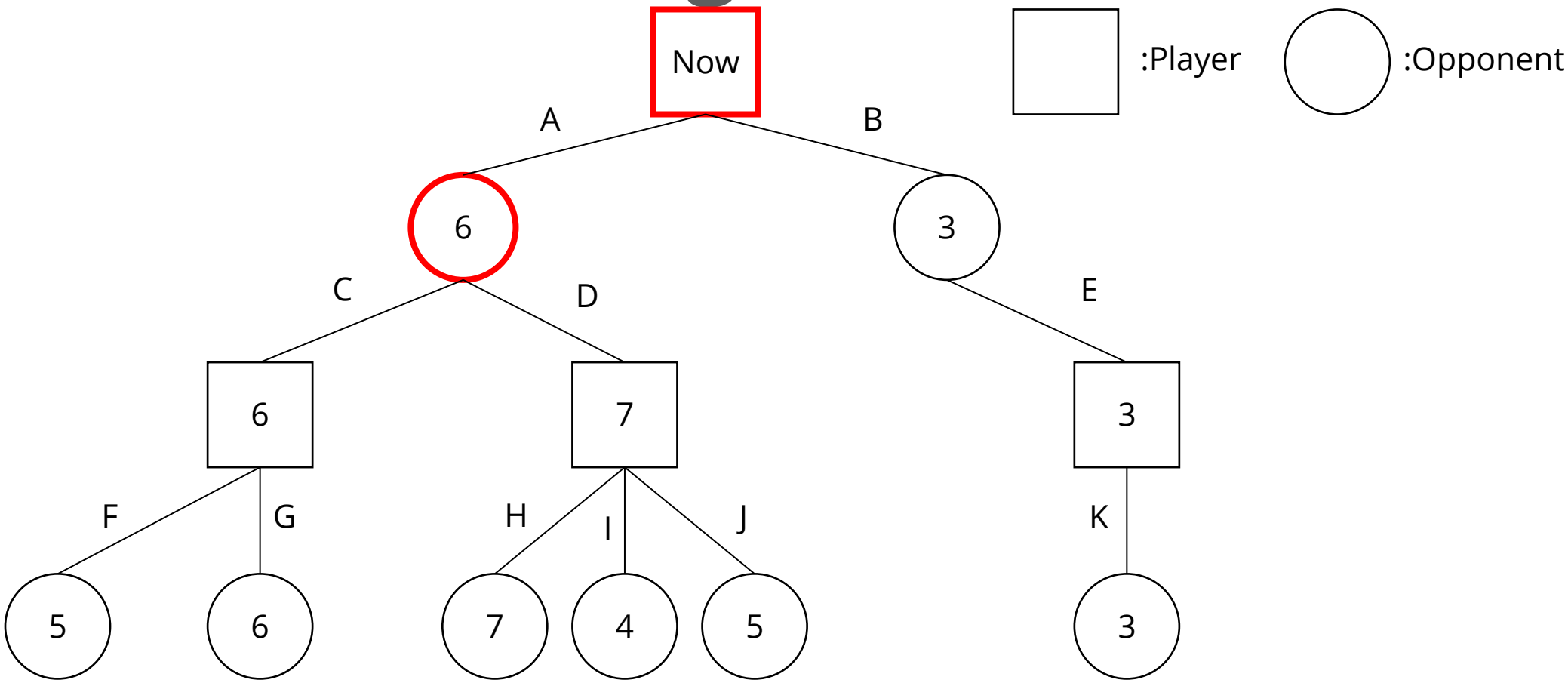
Player picks the largest score



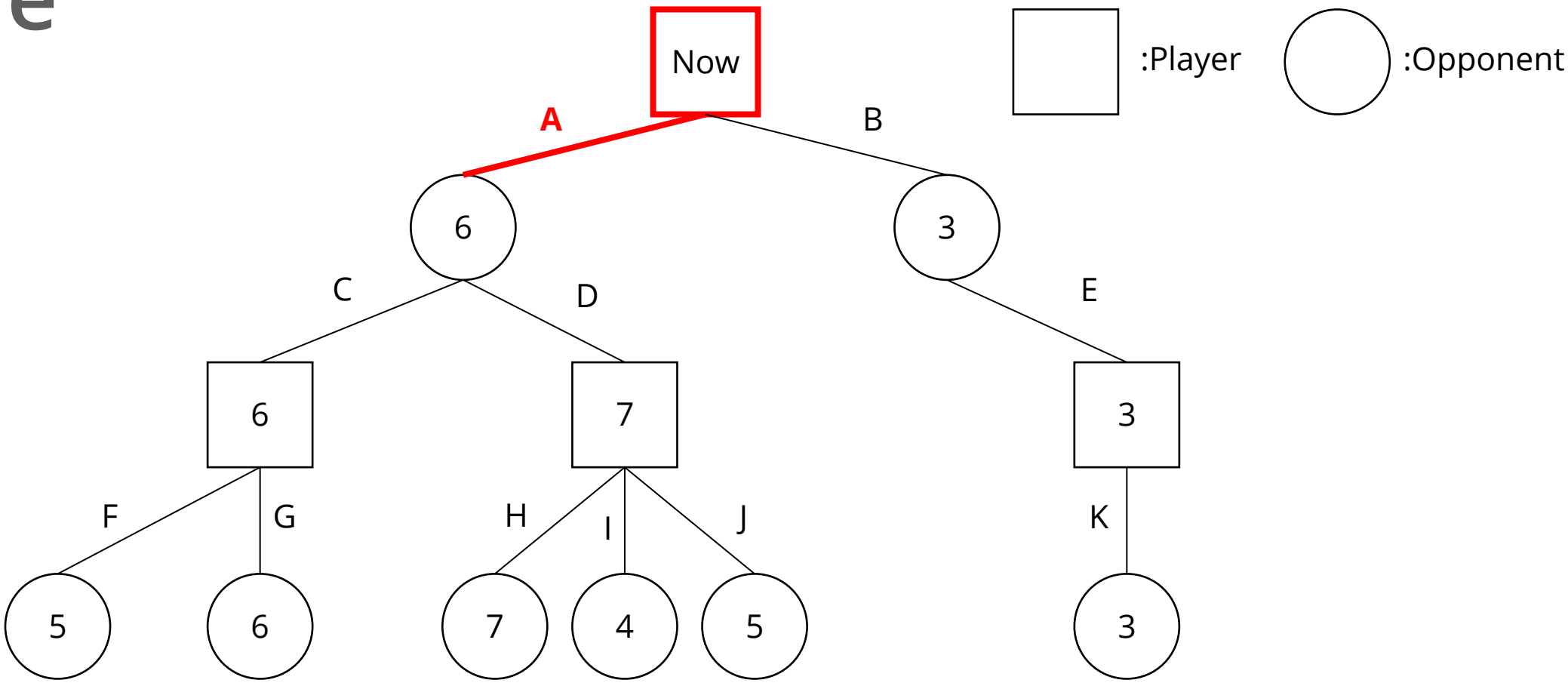
Opponent picks the smallest score



Move A has the largest score



Player picks move A to be the next move



Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

Alpha-Beta Pruning

- By Minimax, we can simulate our opponent's moves and pick a move with minimum risk and maximum value
- Looking forward to more steps that may improve the policy
- However, the size of the search tree may drastically increase with the increase in search depth

Alpha-Beta Pruning

- Since we only have limited time, if we hope to increase search depth, we must optimize the search process
- There are many branches in the minimax process that are not related to the result
- We can try to “prune” these branches to improve efficiency
- The Alpha-Beta Pruning is the improved version of the Minimax method, which eliminates some unnecessary branches

Alpha-Beta Pruning Pseudocode

```
function alphabeta(node, depth, α, β, maximizingPlayer) is
    if depth = 0 or node is a terminal node then
        return the heuristic value of node
    if maximizingPlayer then
        value := −∞
        for each child of node do
            value := max(value, alphabeta(child, depth − 1, α, β, FALSE))
            α := max(α, value)
            if α ≥ β then
                break (* β cutoff *)
        return value
    else
        value := +∞
        for each child of node do
            value := min(value, alphabeta(child, depth − 1, α, β, TRUE))
            β := min(β, value)
            if β ≤ α then
                break (* α cutoff *)
        return value
```

Source:
https://en.wikipedia.org/wiki/Alpha%E2%80%93beta_pruning

Alpha-Beta Pruning

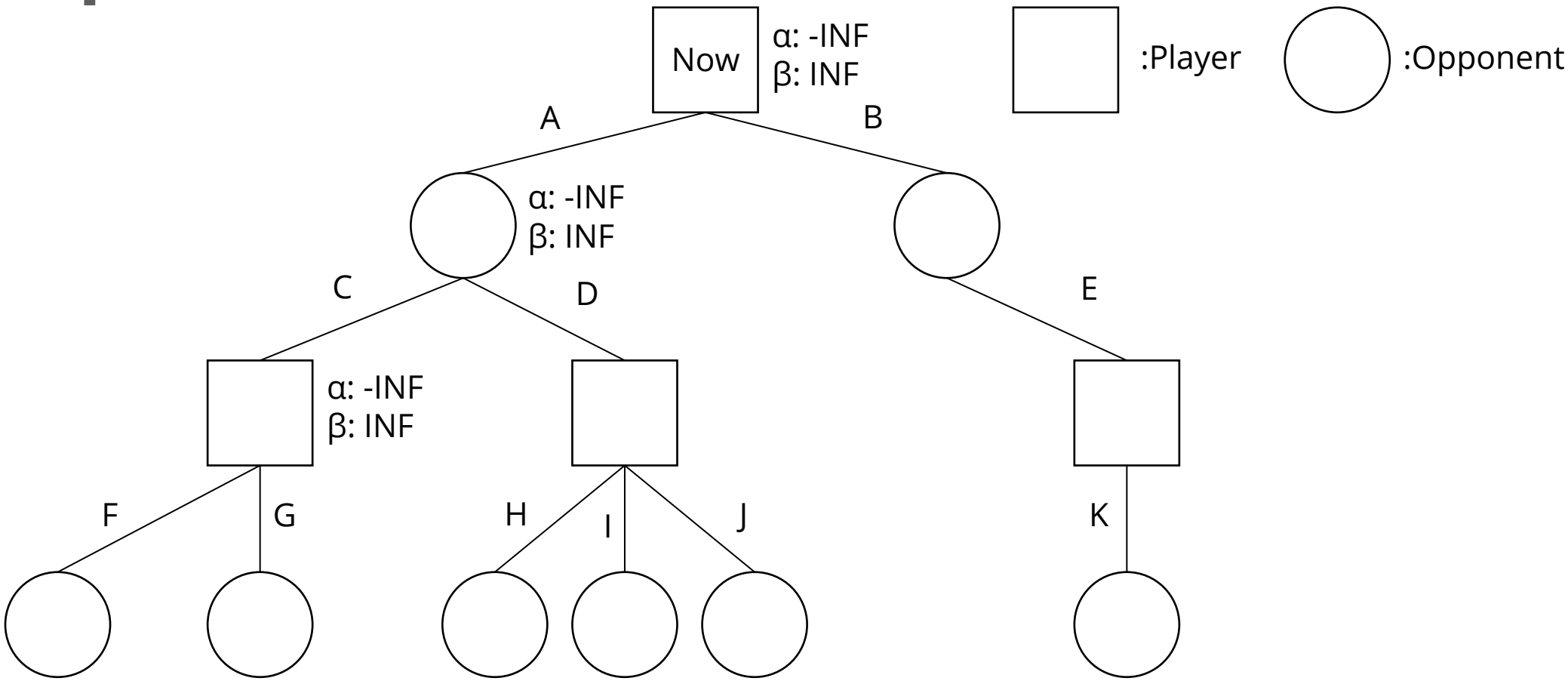
- Alpha: the maximum score that the player is assured of in the current search process

- Beta: the minimum score that the opponent is assured of in the current search process

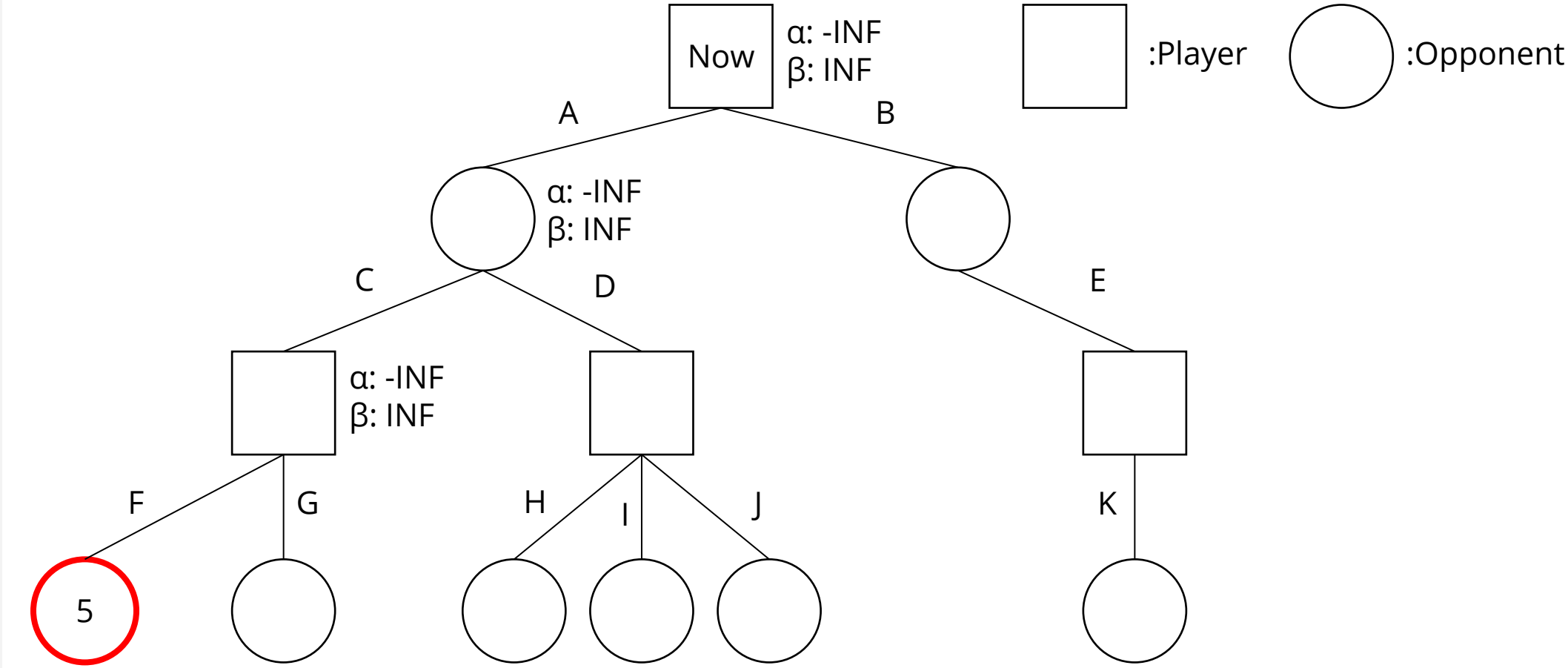
Alpha-Beta Pruning

- If alpha >= beta on a player node, we can stop searching on this branch
- In this situation, the player will return a value >= beta on this branch
- However, the opponent already has a better choice (beta)
- Thus, no matter the later discovered value on this branch, the opponent will not pick this branch
- We can “prune” this branch since it will not affect the result
- We can also stop searching if beta <= alpha on an opponent node

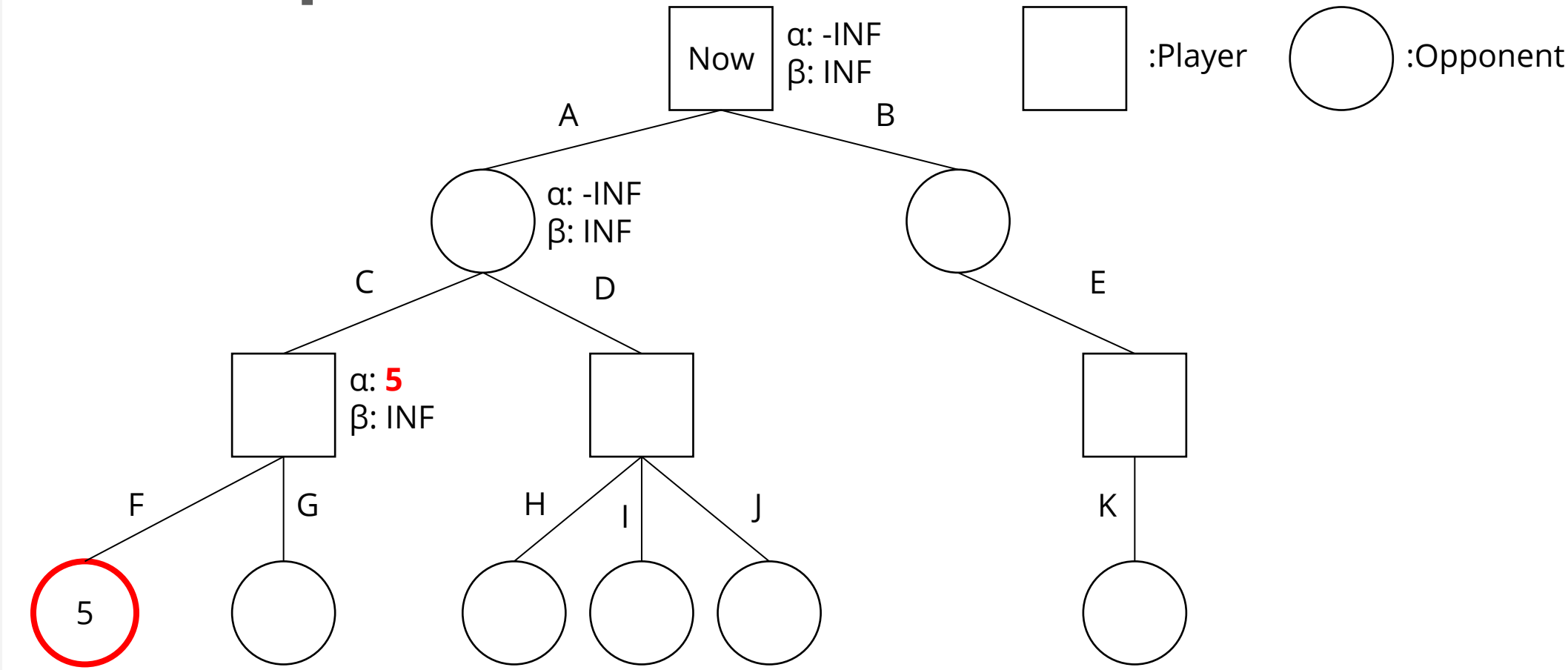
Example


Now
α: -INF
β: INF
:Player
:Opponent
A
B
α: -INF
β: INF
C
D
E
α: -INF
β: INF
F
G
H
I
J
K

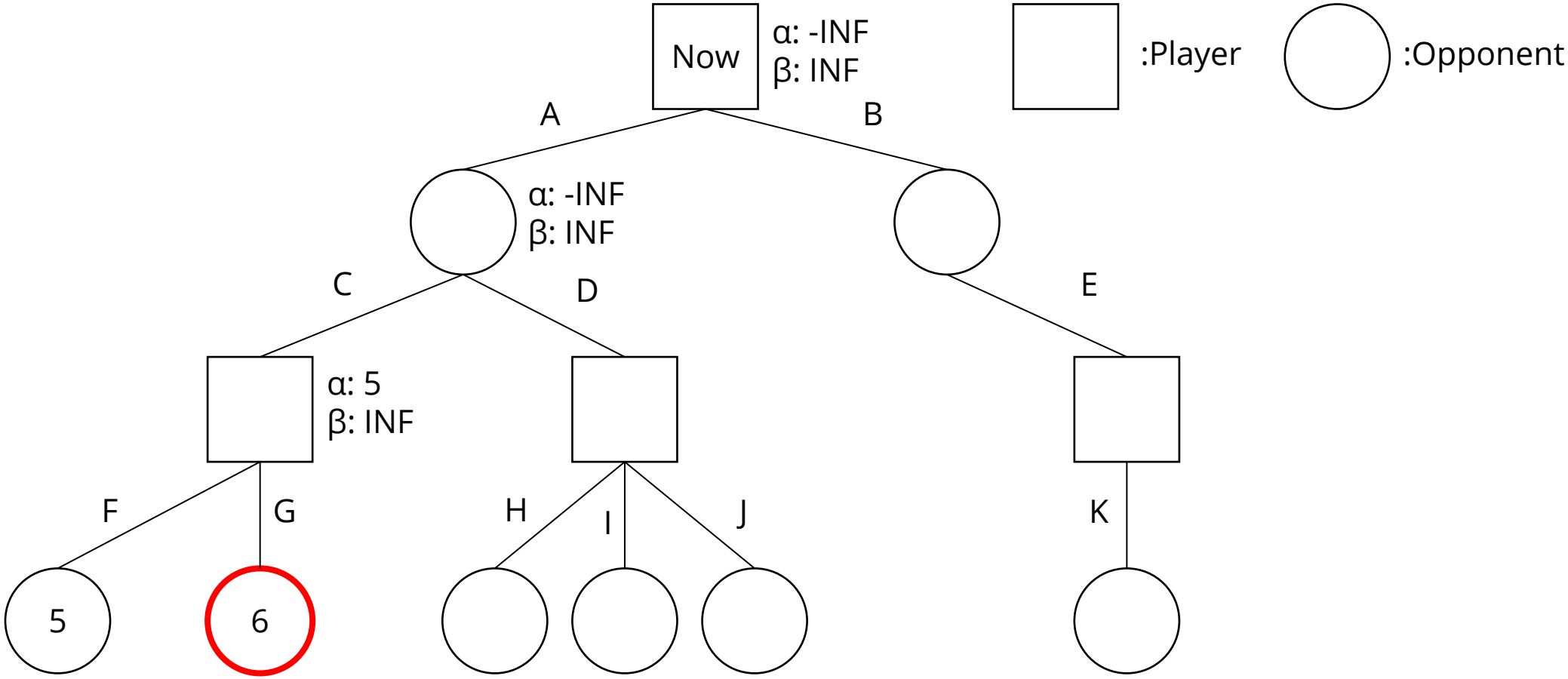
Evaluate score at leaves



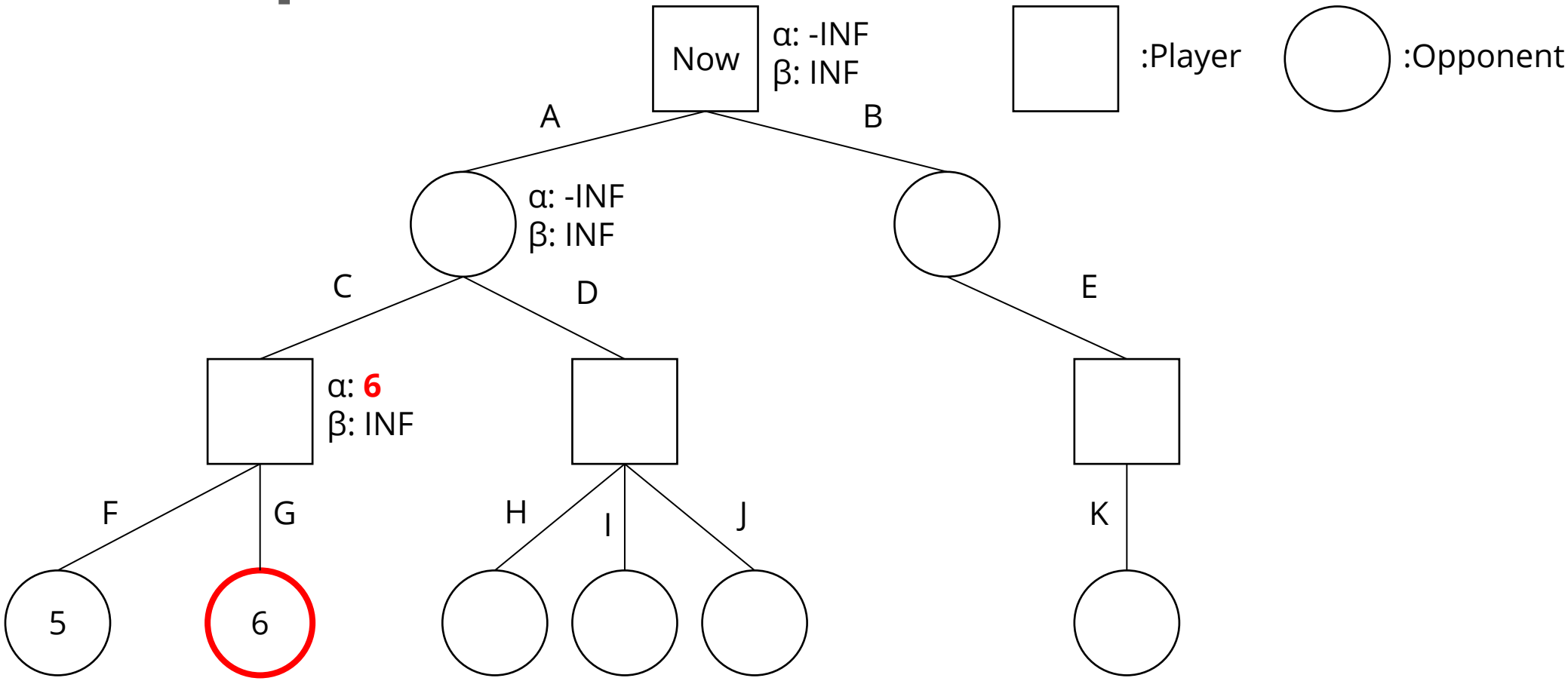
Update alpha



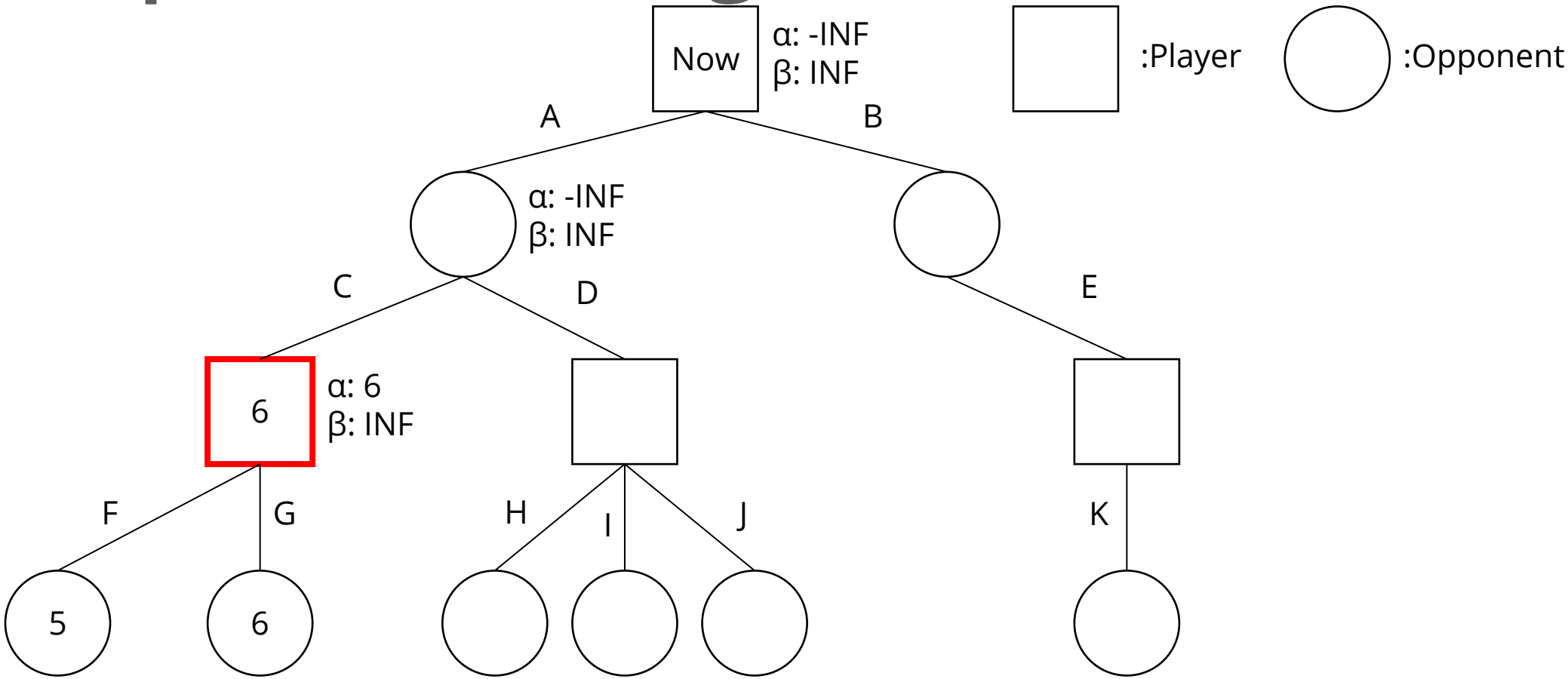
Evaluate score at leaves



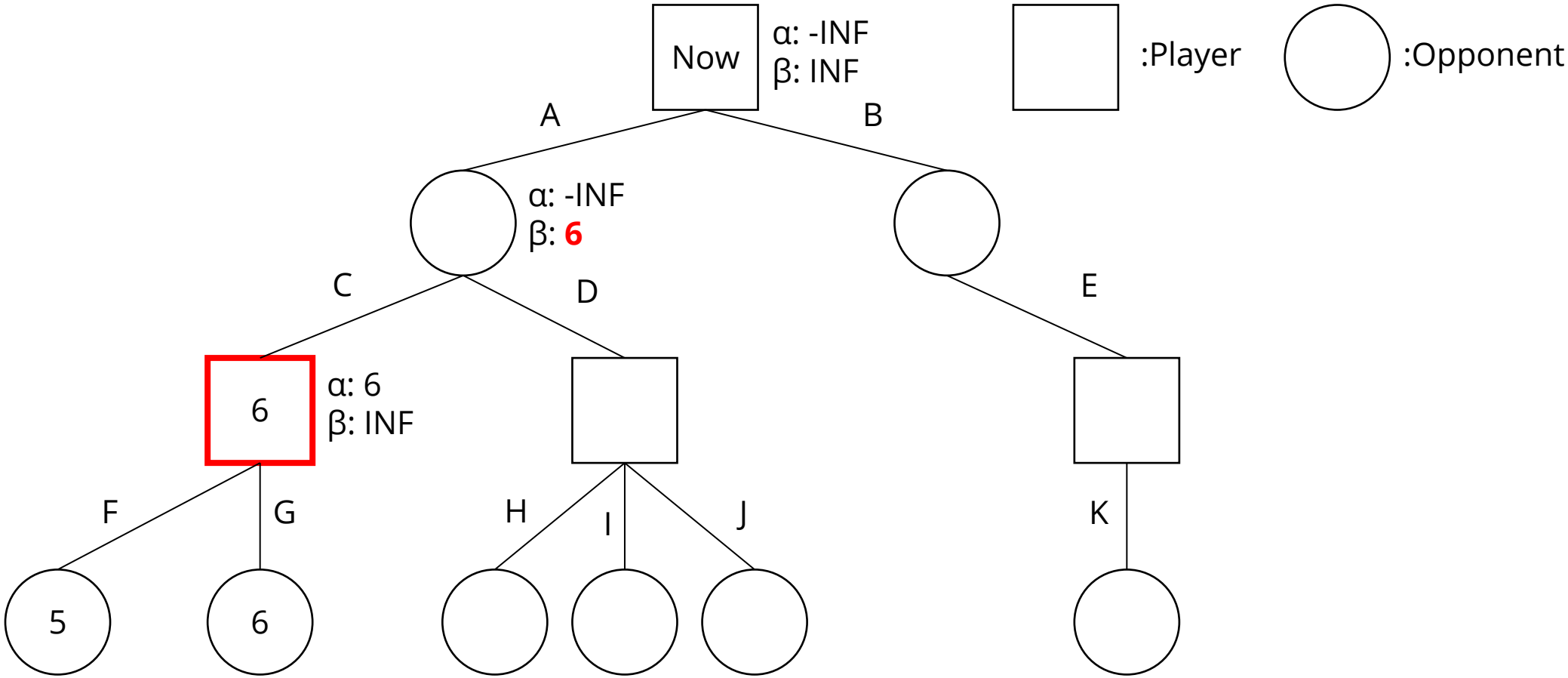
Update alpha



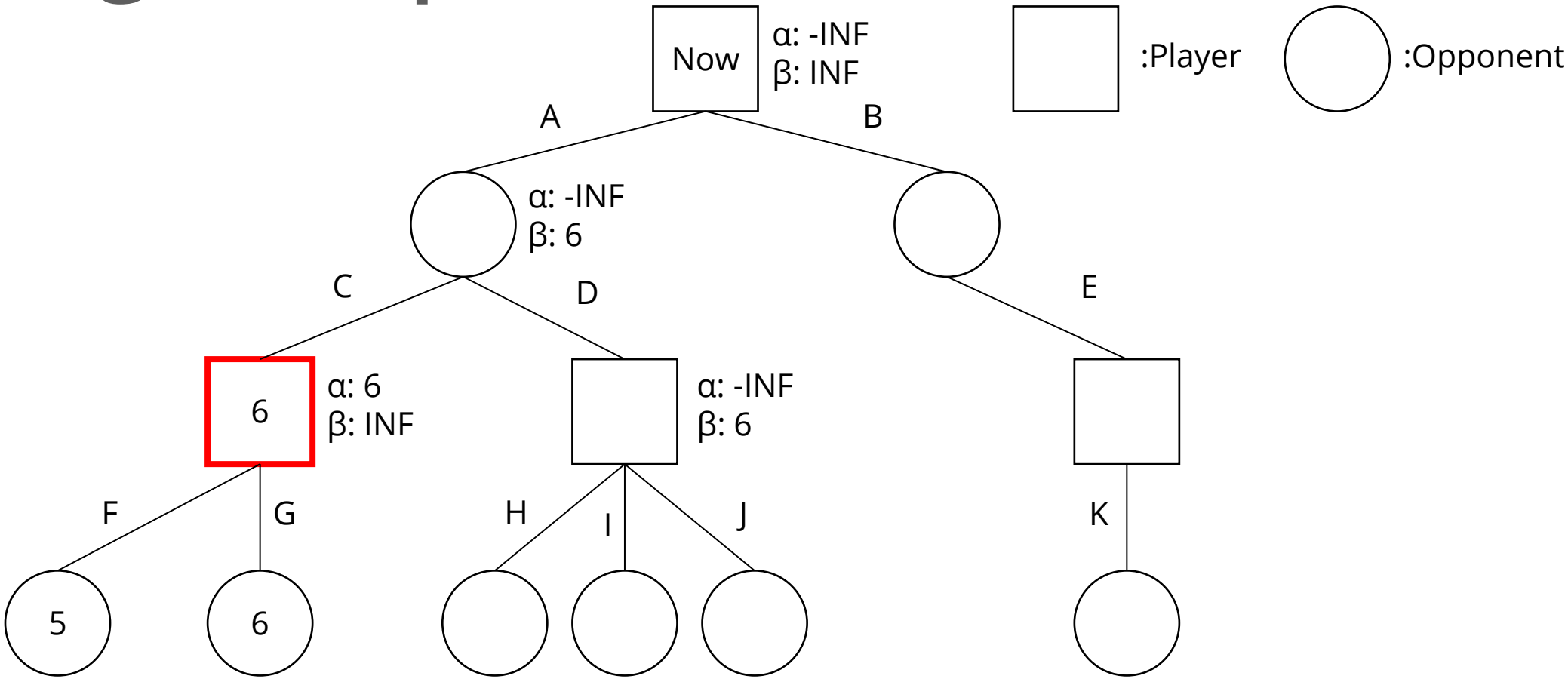
Player picks the largest score



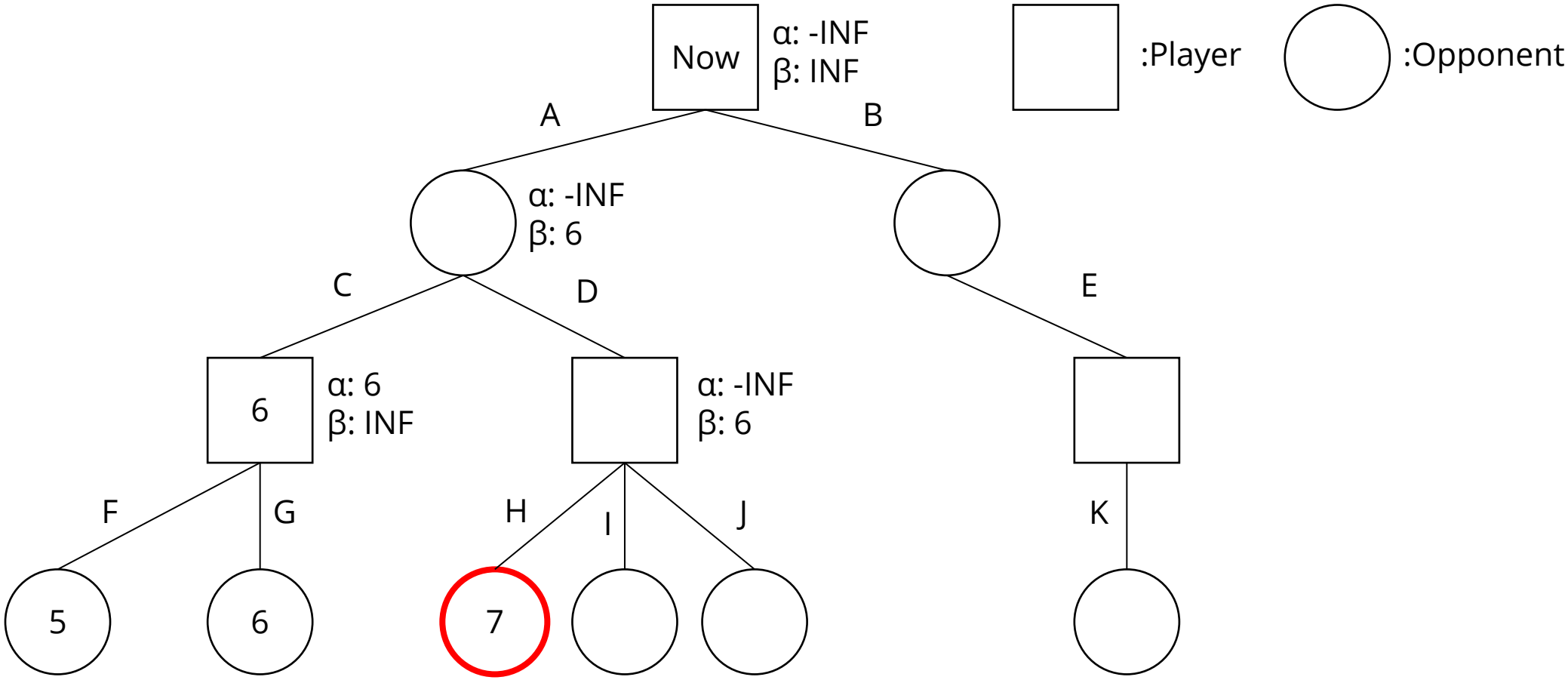
Update beta


Now
α: -INF
β: INF
:Player
:Opponent
A
B
α: -INF
β: 6
C
D
E
6
α: 6
β: INF
F
G
H
I
J
K
5
6

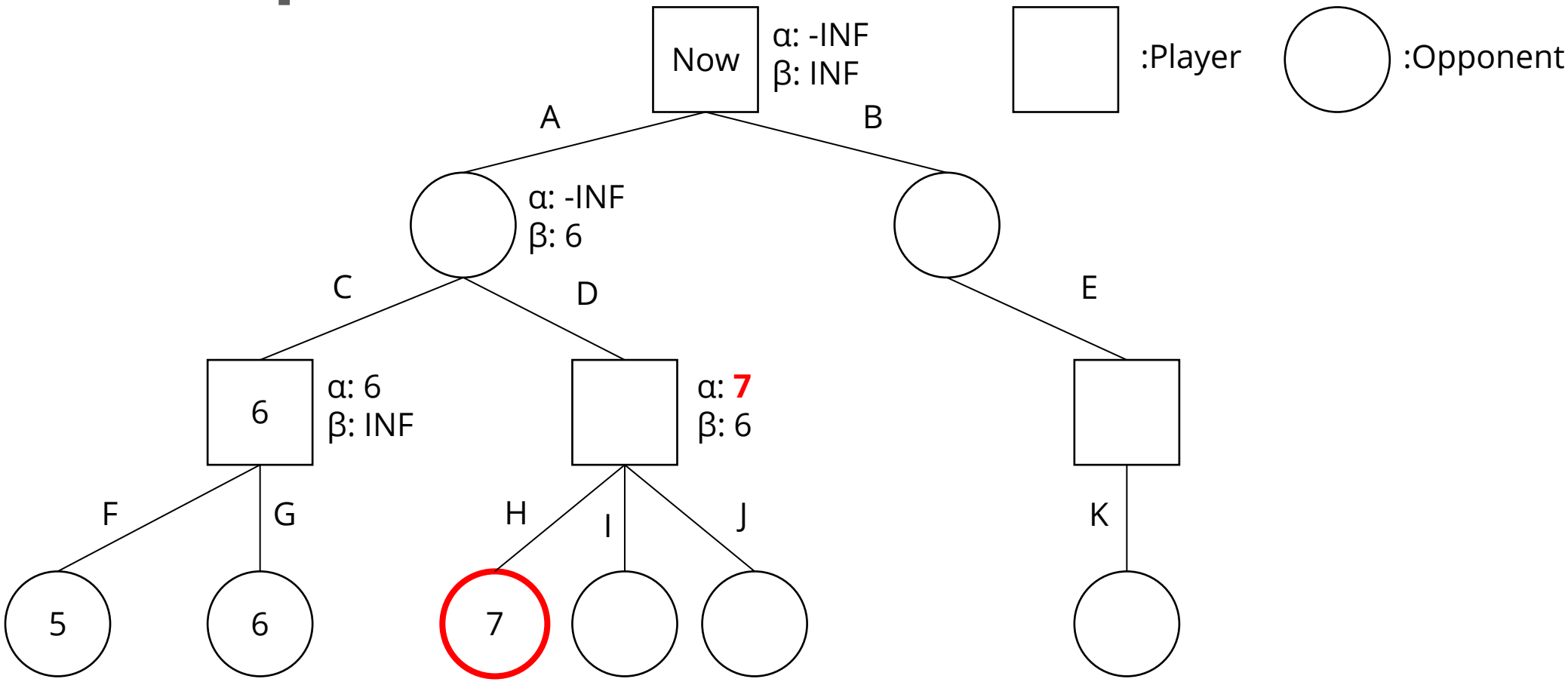
Propagate alpha and beta values



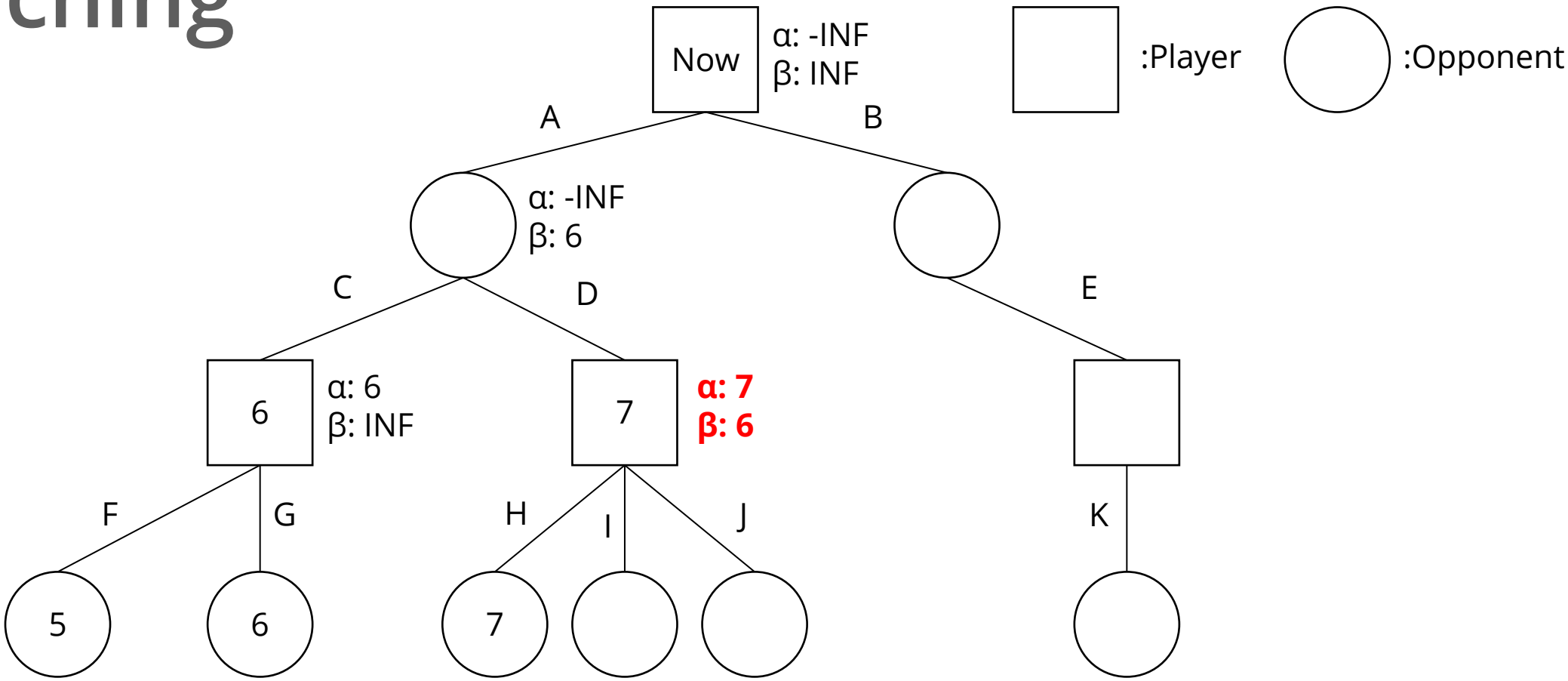
Evaluate score at leaves



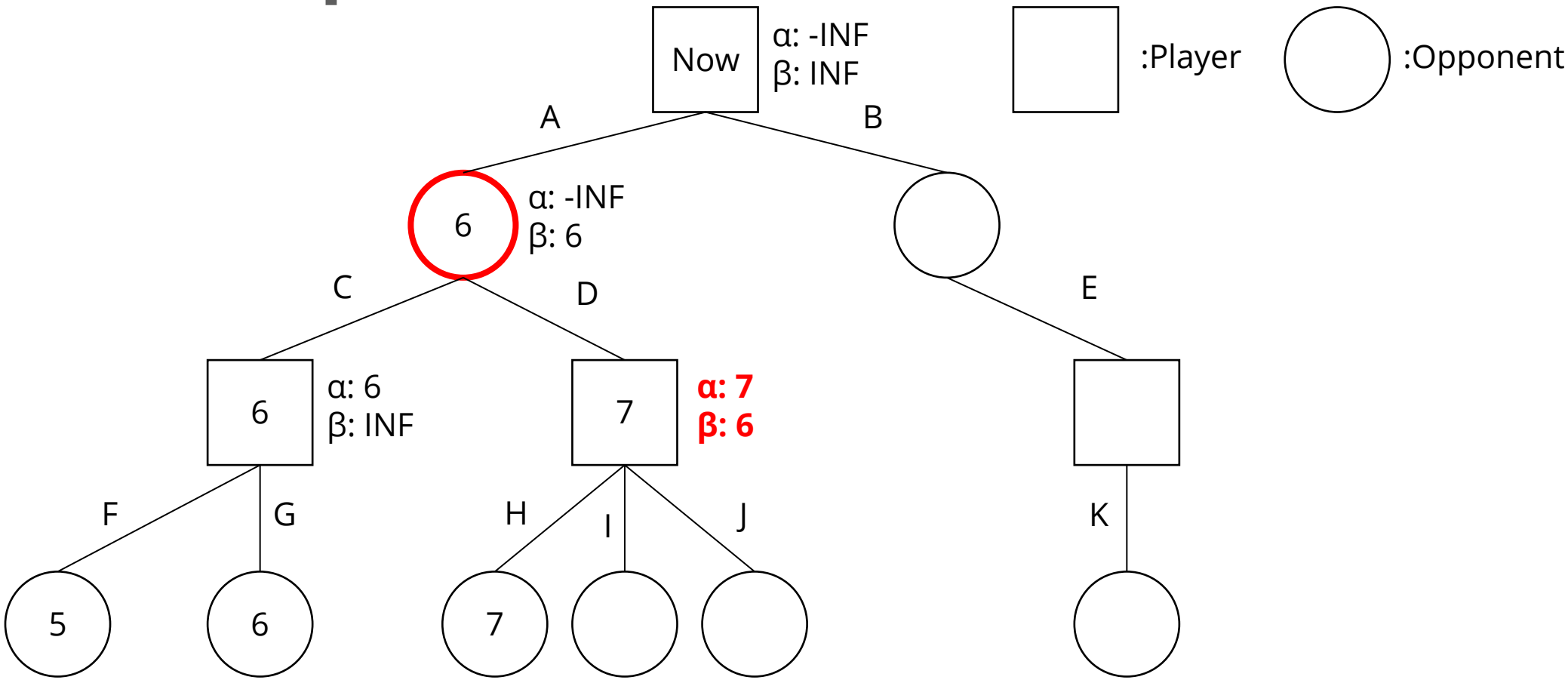
Update alpha



Alpha >= beta in a player node, stop searching



Opponent picks the smallest score



Alpha-Beta Pruning

- In the example above, we use the same search tree as Minimax
- By pruning, we eliminate branches I and J
- However, we still get the same result on branch A
- Alpha-Beta Pruning can effectively speed up the process while maintaining the same result

Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

How to design your AI

- You can refer to the random.cpp/hpp in the src/policy folder
- Design your state value function in state.cpp to evaluate the board
- Implement Minimax method and use your value function in the search process
- Run Minimax and decide which move to output

Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

Package

- You don’t need to implement all the thing by yourself, we will provide some useful utilities and then you can focus on state value function and the tree search algorithm.
- You will get:
  - Game runner
  - State class
    - a native method to get all legal actions
    - Generate new state based on action and state
    - You need to impl state value function by your self
  - An example player (with random choose policy)

Package – How to run it

- Despite the source files, you will also get some additional files:
  - Makefile – help you to compile the project
  - .gitignore – if you need to push your code to github, this can help you ignore some files when you push it
- You can modify your Makefile, but you should make sure it can compile on TAs environment.
- With Makefile and make utils (more details in environment setup document), you can compile your code with make

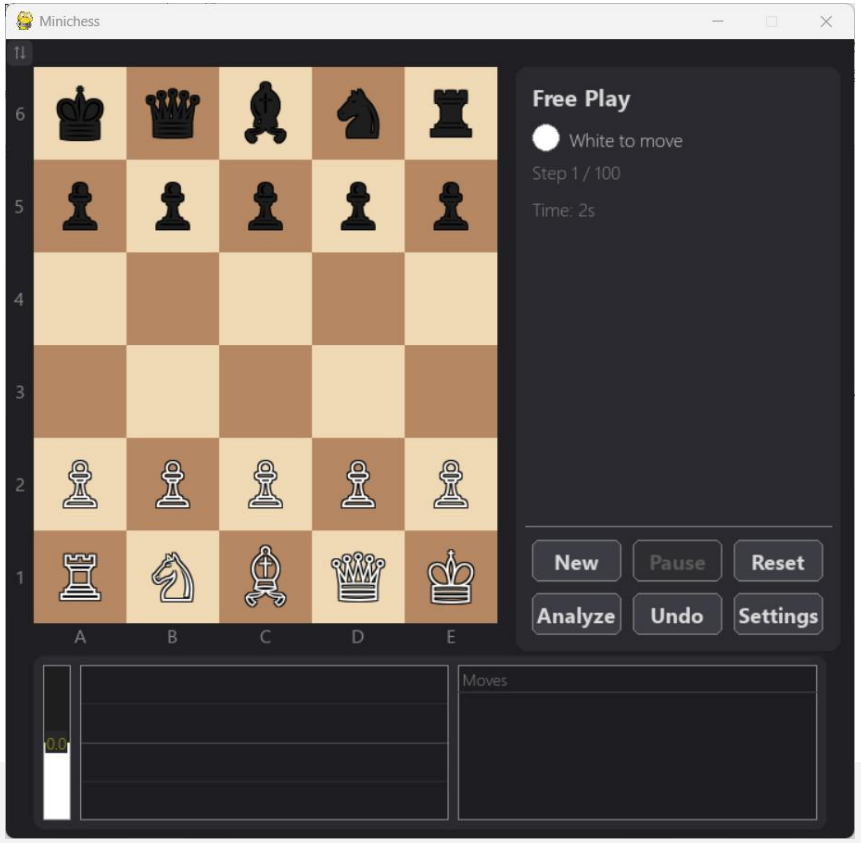
```
$ make
g++ --std=c++2a -Wall -Wextra -Wpedantic -g -O3 -march=native -Isrc/games/minichess -Isrc/state -Isrc -o build/minichess-ubgi src/games/minichess/state.cpp src/policy/minimax.
cpp src/policy/random.cpp src/ubgi/ubgi.cpp
g++ --std=c++2a -Wall -Wextra -Wpedantic -g -O3 -march=native -Isrc/games/minichess -Isrc/state -Isrc -o build/minichess-benchmark src/games/minichess/state.cpp src/policy/min
imax.cpp src/policy/random.cpp src/benchmark.cpp
g++ --std=c++2a -Wall -Wextra -Wpedantic -g -O3 -march=native -Isrc/games/minichess -Isrc/state -Isrc -o unittest/build/state_test src/games/minichess/state.cpp src/policy/min
imax.cpp src/policy/random.cpp unittest/state_test.cpp
```

Package – How to run it

- After running the make all command and compile your program successfully, you can use this command to run the game:

```
$ python gui/main.py
pygame 2.6.1 (SDL 2.28.4, Python 3.13.13)
Hello from the pygame community. https://www.pygame.org/contribute.html
[   0.525] INFO  [Main] Main loop started
```

- And it will start running!



Package – State

- And now, you should start your project.
- You can check the state class first
- next_state can generate new state based on a move
- get_legal_actions will generate all legal actions of this state and store them in legal_actions.
- evaluate is the state value function you need to implement

```
class State : public BaseState {
public:
    Board board;
    int score = 0;
    mutable uint64_t zobrist_hash = 0;
    mutable bool zobrist_valid = false;

    State(){}
    State(int player){
        this->player = player;
    }
    State(Board board): board(board){}
    State(Board board, int player): board(board){
        this->player = player;
    }

    int evaluate(
        bool use_kp_eval = true,
        bool use_mobility = true,
        const GameHistory* history = nullptr
    ) override;
    State* next_state(const Move& move) override;
    void get_legal_actions() override;
    void get_legal_actions_naive();
    void get_legal_actions_bitboard();
    std::string encode_output() const override;
    std::string encode_state();
```

Package - Policy

- If this project, you should implement your own policies. And you will get a random policy for example

```cpp
SearchResult Random::search(
    State *state,
    int depth,
    GameHistory& history,
    SearchContext& ctx
){
    (void)history;
    ctx.reset();
    SearchResult result;
    result.depth = 1;

    if(!state->legal_actions.size()){
        state->get_legal_actions();
    }

    auto actions = state->legal_actions;
    if(actions.empty()){
        result.best_move = Move();
        return result;
    }


    int idx = (rand() + depth) % actions.size();


    result.best_move = actions[idx];
    result.score = 0;
    result.nodes = 1;
    result.pv = {result.best_move};
    return result;
}
```

Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

Requirements - code

- In this project you should do these things:
- 1, design your own state value function
- 2, implement MiniMax, AlphaBeta, PVS and quiescence
- 3, utilize your algorithm and state value function to make a strong AI

Requirements - code

- If you are not satisfied with the basic algorithms, you can try some more advanced methods (MCTS, NNUE). However, make sure you can explain how your algorithms work during the demo.

- If you cannot complete the harder algorithms, implementing the basic ones also gives you some score.

Requirements - code

- You will lose immediately if your program outputs an invalid move
- Time limit for each move is 10 seconds, and the memory limit is 4GB
- You can keep output moves within a limited time. Only the last move is used by the game runner.

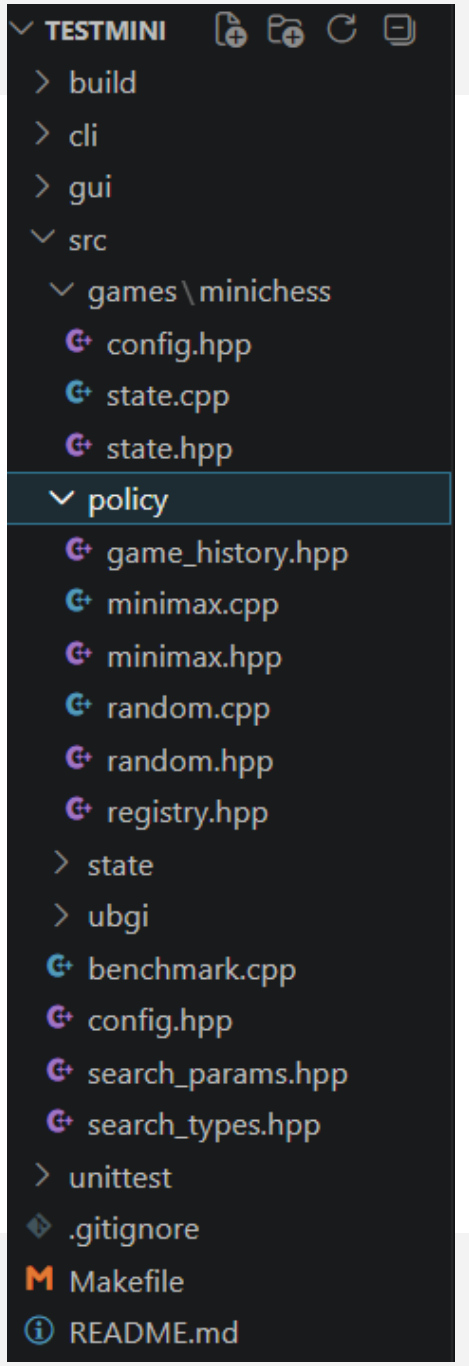
Requirements - code

- You cannot use these things in your code:
  - Third party library (only standard library are acceptable)
  - Inline ASM
  - Multi thread/Multi process
  - Vectorize operation (Like AVX)

Requirements - structure

- Please use C++ and write your program with the structure in the folder

- Make sure to **make a copy** of the policy you want to submit and rename them as “submission.cpp”.



Requirements - structure

- Your program will be compiled in a GNU/Linux environment by:

```
make
g++ --std=c++2a -Wall -Wextra -Wpedantic -g -O3 -march=native -Isrc/games/minichess -Isrc/state -Isrc -o build/minichess
-ubgi src/games/minichess/state.cpp src/policy/minimax.cpp src/policy/random.cpp src/ubgi/ubgi.cpp
g++ --std=c++2a -Wall -Wextra -Wpedantic -g -O3 -march=native -Isrc/games/minichess -Isrc/state -Isrc -o build/minichess
-benchmark src/games/minichess/state.cpp src/policy/minimax.cpp src/policy/random.cpp src/benchmark.cpp
g++ --std=c++2a -Wall -Wextra -Wpedantic -g -O3 -march=native -Isrc/games/minichess -Isrc/state -Isrc -o unittest/build/
state_test src/games/minichess/state.cpp src/policy/minimax.cpp src/policy/random.cpp unittest/state_test.cpp
```

- Please make sure your program can be compiled by the command above with no errors.
  (If you don't change the makefile, just use "make" command and see if there is any error.)

Requirements - Report and Demo

- You should write a report to elaborate on how you design your AI
- If you have implemented NNUE or MCTS, please provide a brief description in your report of how it was implemented, along with code explanations if possible.
- The report is not directly graded, but is your only available reference through the TA demo (You cannot refer to your code in demo)
- You must attend the demo and answer the questions from TA
- The demo date and method will be announced soon

Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

Grading

- The project accounts for 15 points of your total grade
- Design of your state value function => +1 point
- Implement Tree search (Minimax) => +2 point
- Implement Alpha-Beta Pruning => +1 point
- Beat every baseline => +8 points
  (1.5 point for each 4 baselines, 2point if you beat all the baselines with both white and black)
- Implement PVS => +2 point
- Quiescence => +1 point

TODOs in hackathon

Grading - baselines

- We have 4 baselines:
  - Weak MiniMax
  - Strong MiniMax
  - AlphaBeta
  - PVS
- Your program will play with baselines for both white and black.
- If you can get 1win + 1draw (or 2win), you can go to next baselines and get the score.
- If you can beat all the baselines with 8wins, you get final 2 points.

Grading Bonus

- (Bonus) Use version control software => +1 point
  - You need to push your code to github as submission. but if you also use github+git as version control software and has at least 3 commits, you will get this point.
- (Bonus) Class ranking => At most +2 points
  - You can attend the class ranking if you beat all baselines (8wins).
  - Your AI will play against other Ais of your classmates and gain bonus score according to your ranking.
- (Bonus) Beat the visible boss => + 1 point
- (Bonus) Beat the secret boss => + 2 point

Grading Plagiarism

- We will compare the code from both classes and previous years to detect plagiarism.

- If plagiarism is found, all involved submissions will receive a score of 0, regardless of who copied from whom.

Outline

1. Introduction
2. Chess and Mini Chess
3. State Value Function
4. Minimax
5. Alpha-Beta Pruning
6. How To Design Your AI
7. Package
8. Requirements
9. Grading
10. Submission

Submission

- Submit the report to Mini Project 2 繳交區on eeclass

  File name: <student_id>_project2.pdf

  Ex: 114000000_project2.pdf

- Deadline: 6/20 (both report and google form)
- Late submissions within 2 hours will receive a 40% deduction in score.(六折)
- Late submissions more than 2 hours will receive a score of 0.

Happy Coding!